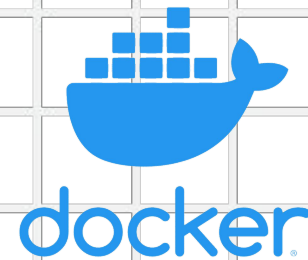
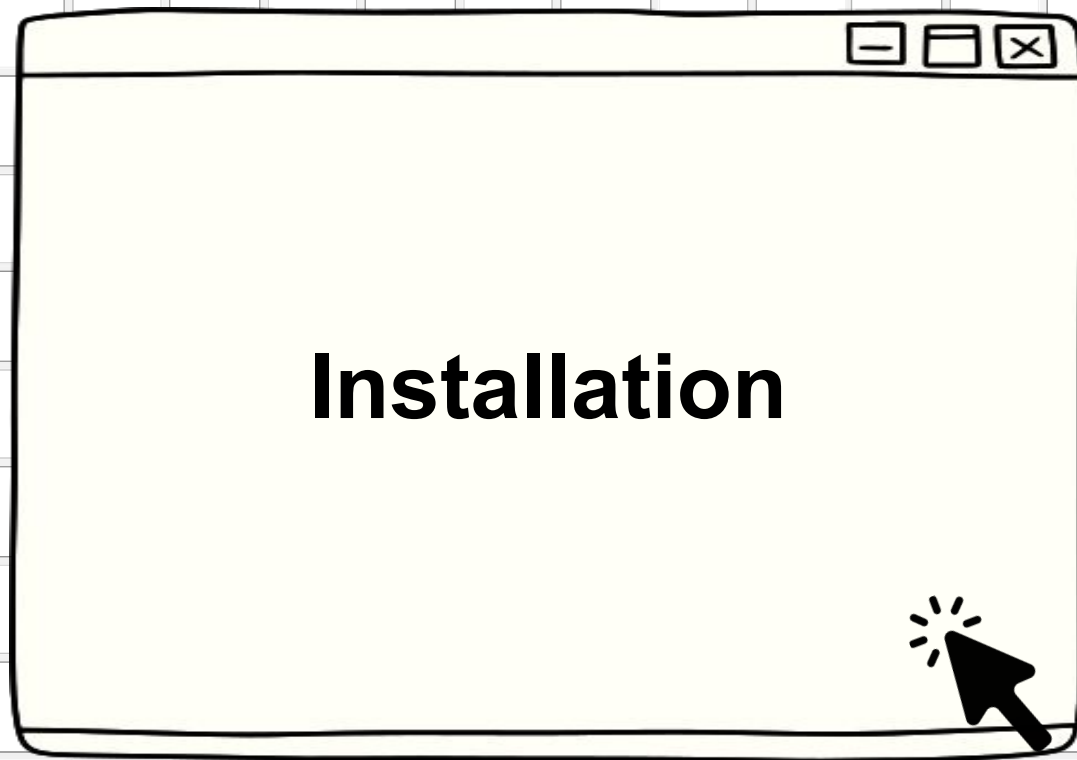
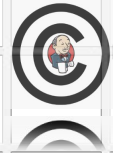
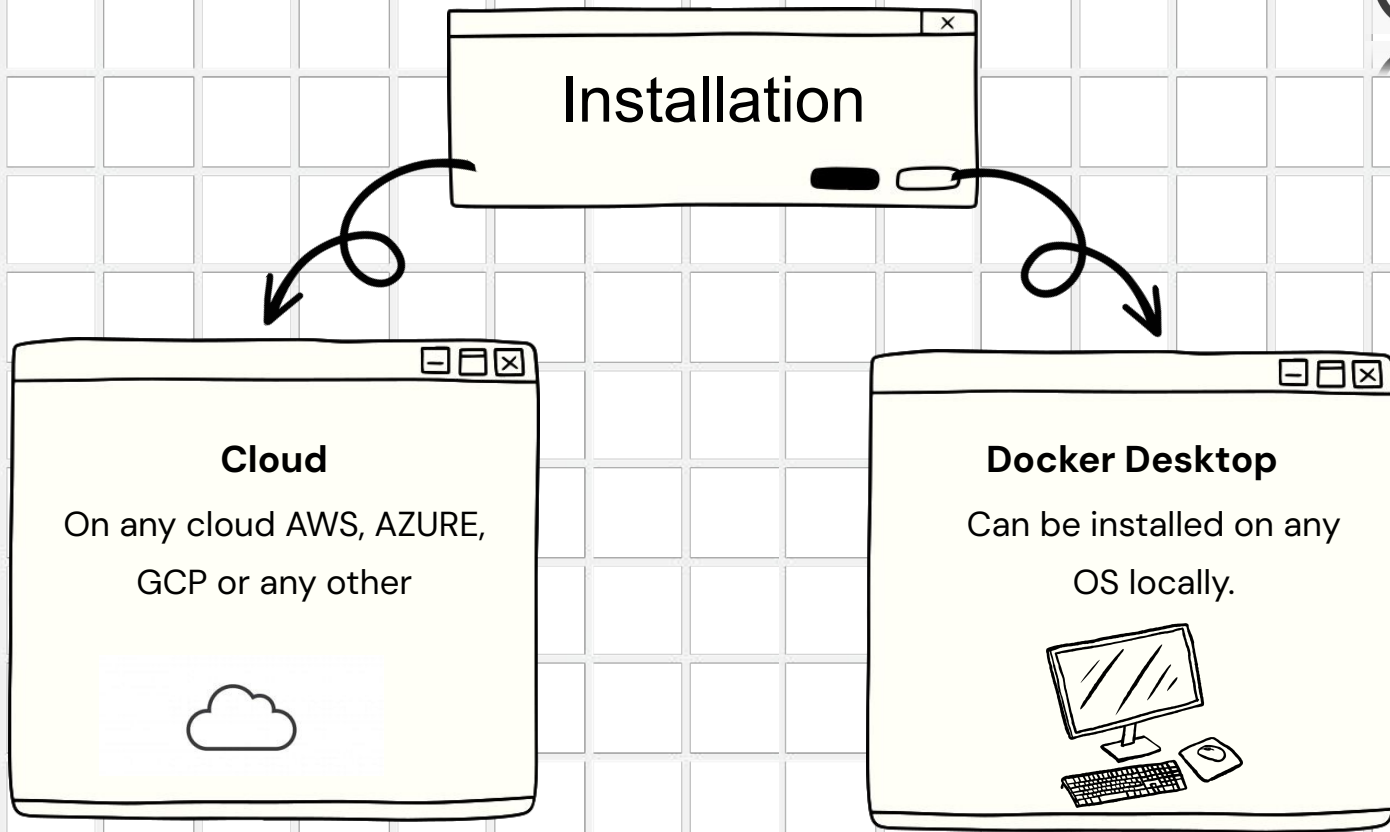
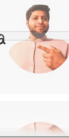


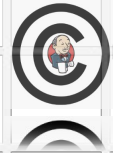
Karan Gupta



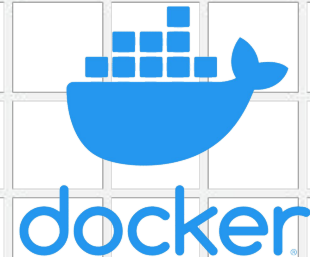
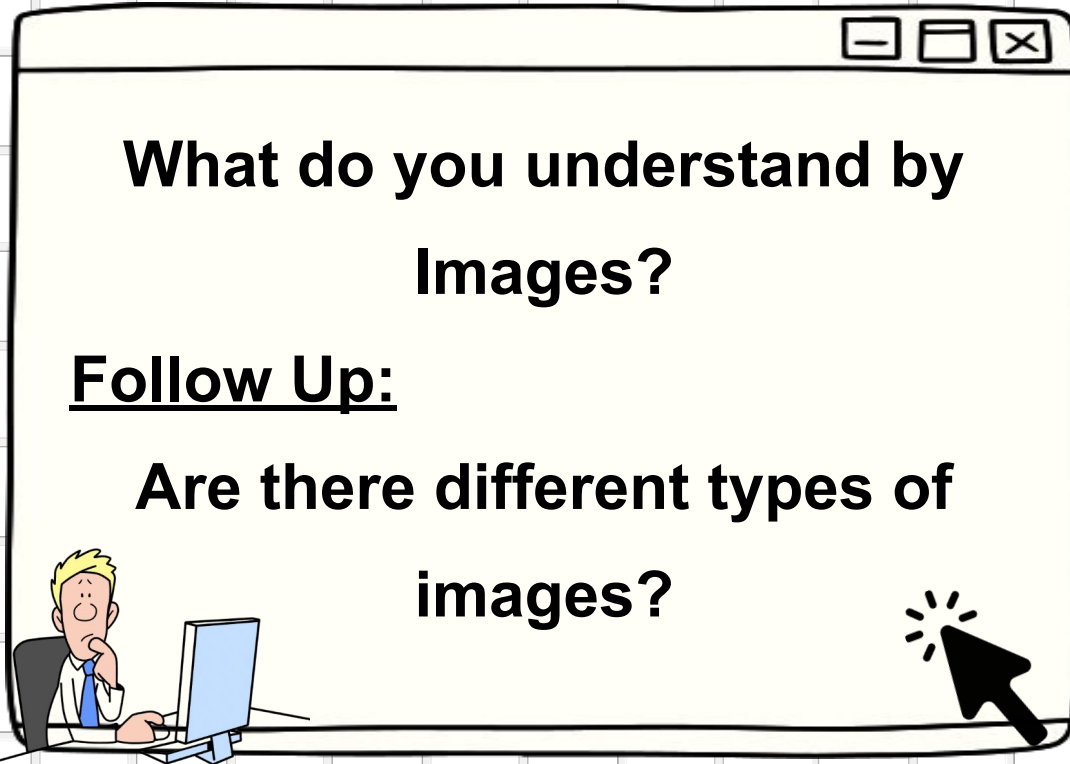


Karan Gupta

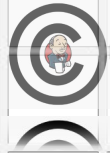




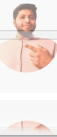
Karan Gupta



Docker Images



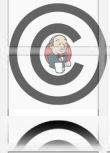
Karan Gupta



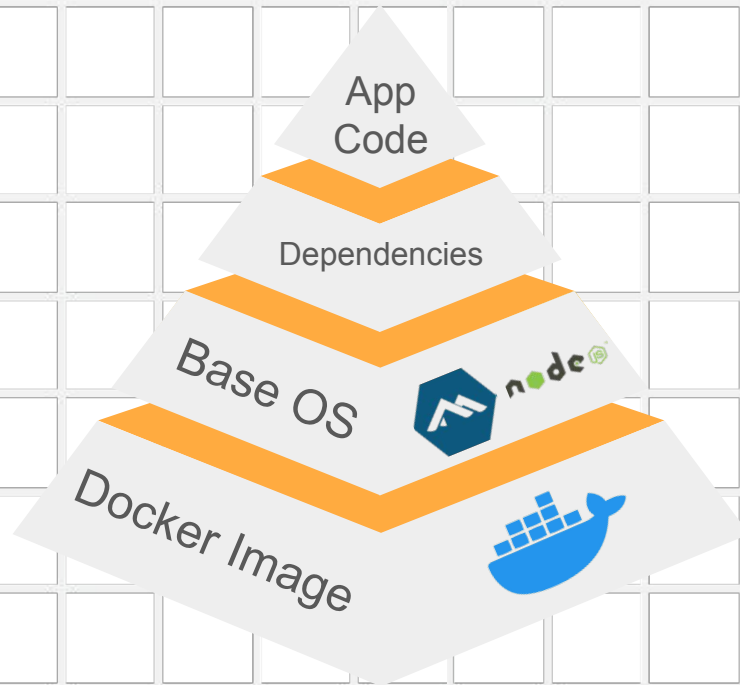
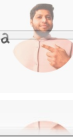
- A **Docker Image** is a lightweight, standalone, executable package that includes everything needed to run a piece of software.
- A **Docker Image** is a **blueprint** for running applications inside containers.

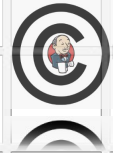
Key Features:

- a. Portable
- b. Stored in **Docker Hub, AWS ECR, or private registries**
- c. Used to launch **Docker Containers**



Karan Gupta





Karan Gupta



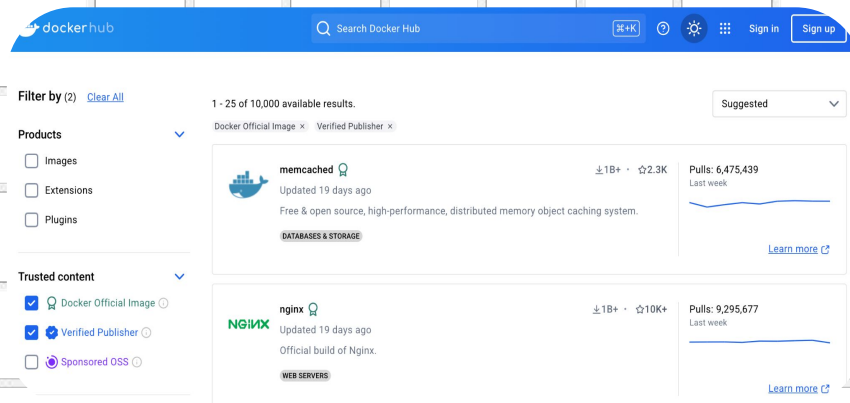
Images

Official Images

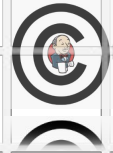
Images created and managed by official publishers on Docker Hub.

Custom Images

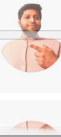
Created as per the requirement.



```
sample
Users > karangupta > Downloads > star > aws trainings > sample > ...
1 # Start from a base image
2 FROM python:3.9
3
4 # Set the working directory
5 WORKDIR /app
6
7 # Copy application code
8 COPY . .
9
10 # Install dependencies
11 RUN pip install -r requirements.txt
12
13 # Define the command to run the app
14 CMD ["python", "app.py"]
15
```

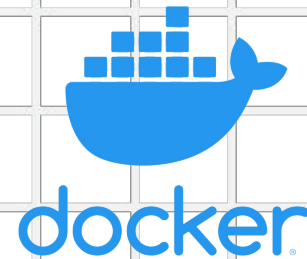


Karan Gupta

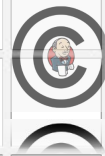


Docker Images - I

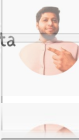
(List, Pull & History)



Docker Images Syntax



Karan Gupta



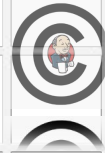
A Docker image follows a specific naming convention:

- Image name = `<registry>/<repository>:<tag>`

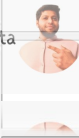


Components:

- `registry` → Optional, points to Docker Hub or private registries.
- `repository` → The name of the image (e.g., `nginx`, `ubuntu`).
- `tag` → Specifies the version (e.g., `latest`, `1.0`, `alpine`). If omitted, `latest` is assumed.



Karan Gupta



List Docker Images

`docker images`

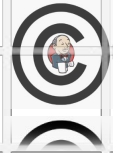
Pull an Image from a Registry

`docker pull <image_name>:<tag>`

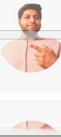
View Image History

`docker history <image_name>`

- `docker pull` → Downloads Image from Registry
- `docker images` → Lists Images Locally
- `docker history` → Shows Image Layers

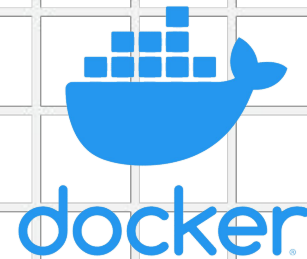


Karan Gupta

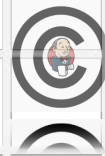


Docker Images - II

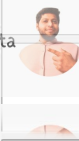
(Tag, Remove & Inspect)



Docker Images Commands



Karan Gupta



- **Tag a Docker Image** – Assign a new name or version to an existing image.

```
docker tag <source_image>:<source_tag> <target_image>:<target_tag>
```

Example:

```
docker tag myapp:1.0 myrepo/myapp:latest
```

- **Remove a Docker Image** – Deletes an image from local storage.

```
docker rmi <image_name>:<tag>
```

Example:

```
docker rmi myapp:1.0
```



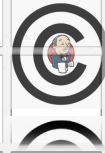
- **Inspect a Docker Image** – Displays detailed metadata about an image, including:

- ❑ Image ID
- ❑ Creation date
- ❑ Size
- ❑ Environment variables
- ❑ Configuration settings

`docker inspect <image_name>`

Example:

`docker inspect nginx`

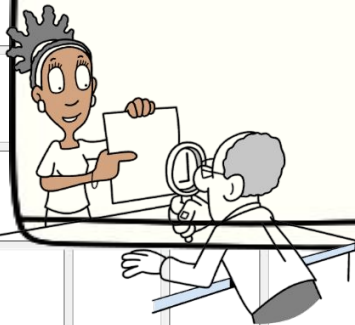
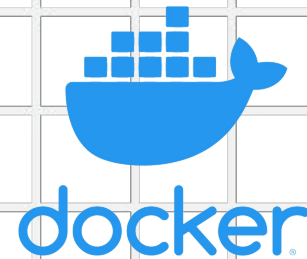


Karan Gupta

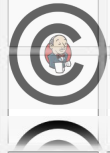


Docker Images - III

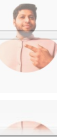
(Version, Search, Help & Manual)



Docker Images Commands



Karan Gupta



- **Check Docker Version** – Displays the installed Docker version.

```
docker <command> <source_image_name>:version
```

Example:

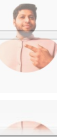
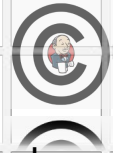
```
docker pull jenkins/jenkins:its-jdk21
```

- **Search for an Image in Docker Hub** – Finds available images from **Docker Hub**

```
docker search <image_name>
```

Example:

```
docker search nginx
```



- **Get Help for Docker Commands** - Get help on usage of the docker commands

`docker help <command>`

Example:

`docker help pull`

- **View Docker Manual Pages** - Opens the full **manual** for Docker, detailing all commands and options.

`man docker`



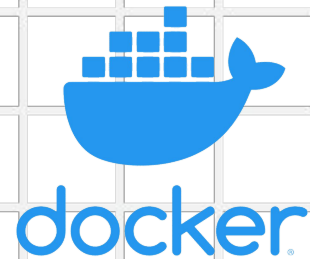
Karan Gupta



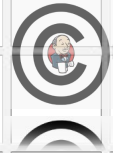
What are Containers?

OR

**Explain the process of
Containerization.**



Containers



Karan Gupta



A **container** is a lightweight, standalone, and executable package that includes everything needed to run an application.

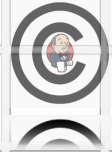
Key Features:

- **Portable** – Runs consistently across environments
- **Isolated** – Works independently from other containers
- **Lightweight** – Shares the host OS kernel instead of using a full VM

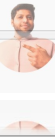
How it Works:

1. **Developer** builds an application and packages it in a container.
2. **Container Runtime** runs it on any system with Docker installed.
3. **Multiple Containers** can run on the same host, each isolated from the other.

Containerization



Karan Gupta



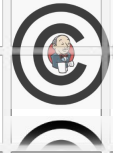
Containerization is the process of packaging an application and its dependencies into a container.

Benefits:

- **Consistency:** Works the same across development, testing, and production.
- **Efficiency:** Uses fewer resources than traditional virtual machines.
- **Scalability:** Easily deploy and scale applications in microservices architectures.
- **Rapid Deployment:** Faster build, test, and deployment cycles.

Popular Tools:

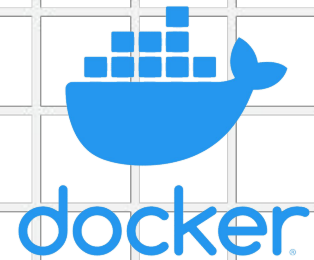
- Docker
- Kubernetes (for orchestration)



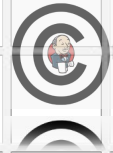
Karan Gupta



**What is the difference
between VM & Containers?**



Virtual Machine



Karan Gupta

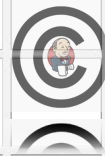
A Virtual Machine is an emulation of a physical computer. It runs a full operating system (OS) on top of virtualized hardware, managed by a hypervisor.

Key Components:

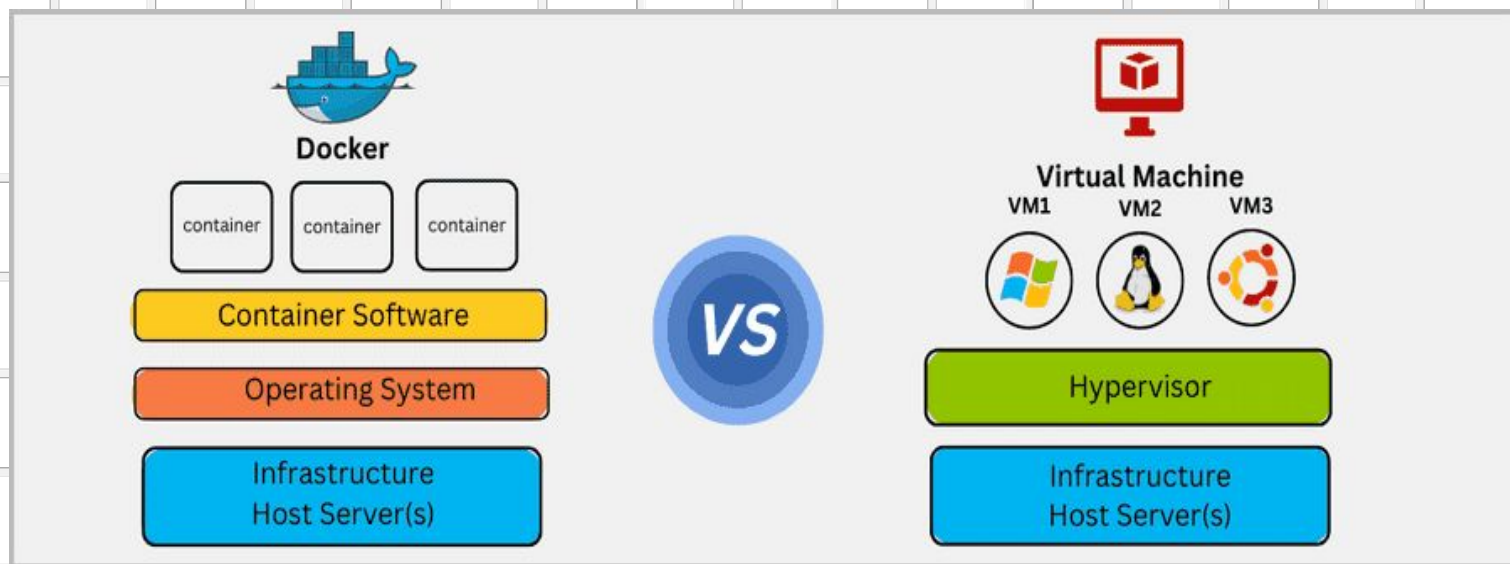
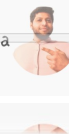
- Guest OS (separate from host)
- Virtual hardware (CPU, memory, disk)
- Managed by a **Hypervisor** (e.g., VMware, Hyper-V, KVM)

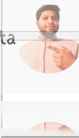
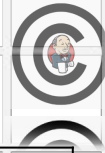
Use Cases:

- Running multiple OS types on one machine
- Legacy application support
- Strong isolation for security

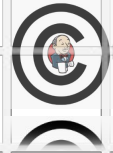


Karan Gupta





Feature	Virtual Machines	Containers
OS	Each VM runs its own OS	Share host OS kernel
Performance	Slower (more overhead)	Faster (lightweight)
Isolation	Stronger (hardware-level)	Moderate (process-level)
Boot Time	Minutes	Seconds
Resource Usage	High	Low
Portability	Less portable	Highly portable
Use Case	Monolithic apps, legacy systems	Microservices, cloud-native apps

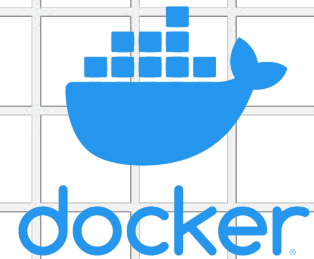


Karan Gupta

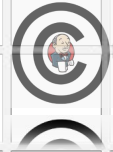


Follow-Up:

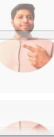
**Give real-time scenario
where both are applicable?**



Real Time Scenario



Karan Gupta



A financial services company is running a **hybrid cloud architecture** to balance legacy systems and modern applications.

Use of VMs:

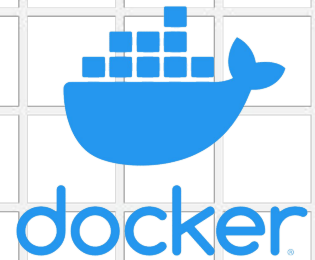
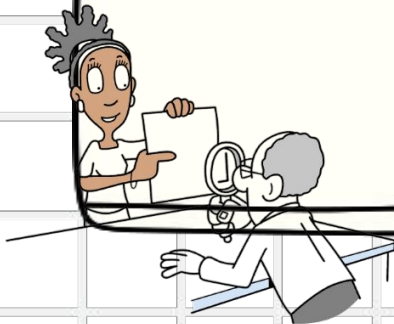
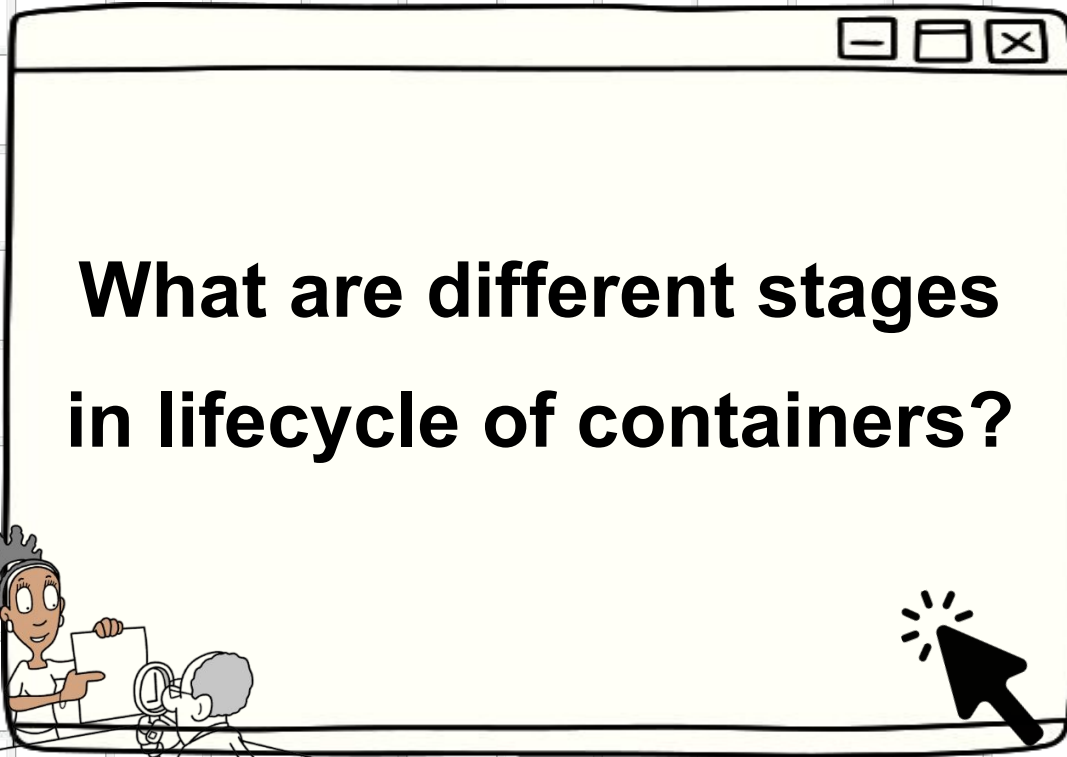
- Hosts **legacy core banking software** that requires full OS isolation and runs only on specific environments.
- Provides **strong security boundaries** between critical systems.

Use of Containers:

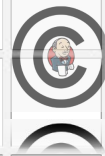
- Runs **microservices** like customer notifications, fraud detection, and dashboards.
- Enables **rapid deployment, auto-scaling, and CI/CD pipelines** using Kubernetes and Docker.
- Deployed across **on-premises and cloud** environments for flexibility.



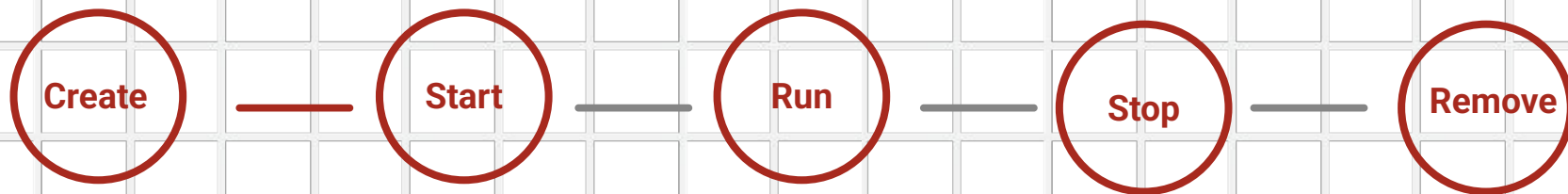
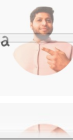
Karan Gupta



Lifecycle of Containers



Karan Gupta



- **Create**

- A container is created from a Docker image using:
`docker create <image>`

- **Start**

- The container is started and begins running:
`docker start <container_id>`

- **Running**

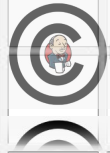
- The container is actively executing processes.

- **Pause (Optional)**

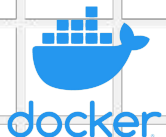
- Temporarily suspends all processes:
`docker pause <container_id>`

- **Unpause (Optional)**

- Resumes paused processes:
`docker unpause <container_id>`



Karan Gupta



6. Stop

- Gracefully stops the container:

```
docker stop <container_id>
```

7. Kill (Optional)

- Immediately stops the container (forcefully):

```
docker kill <container_id>
```

8. Restart (Optional)

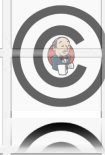
- Stops and then restarts the container:

```
docker restart <container_id>
```

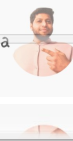
9. Remove

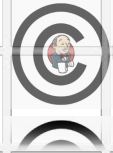
- Deletes the container from the system:

```
docker rm <container_id>
```



Karan Gupta





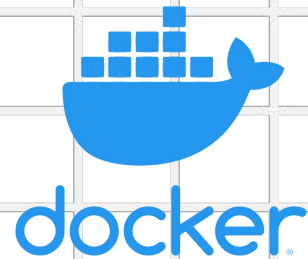
Karan Gupta



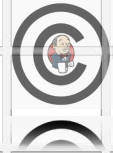
**Explain me the architecture of
Docker.**

OR

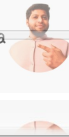
**What do you understand by
docker client server architecture?**



Docker Architecture



Karan Gupta



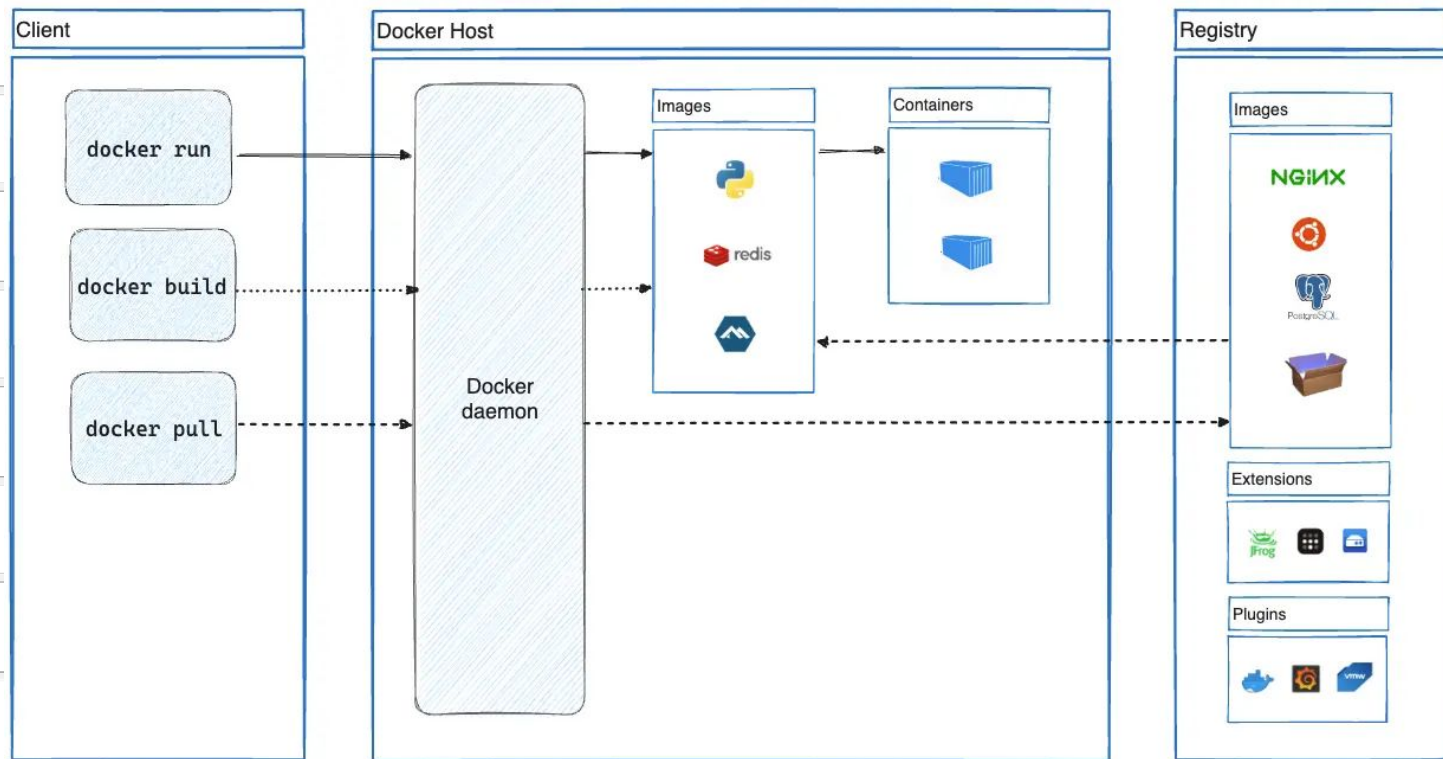
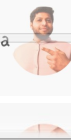
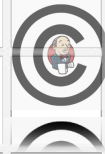
- Docker uses a client-server architecture.
- The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers.

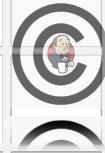
1. Docker Client

- The user interface for Docker.
- Sends commands (`docker build`, `docker run`, etc.) to the Docker daemon.

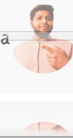
2. Docker Daemon (dockerd)

- Core service that runs on the host.
- Builds, runs, and manages Docker containers.
- Listens for API requests and handles container lifecycle.





Karan Gupta



3. Docker Images

- Read-only templates used to create containers.
- Built from Dockerfiles.

4. Docker Containers

- Running instances of Docker images.
- Lightweight, isolated, and include app + dependencies.

5. Docker Registry (e.g., Docker Hub)

- Storage for Docker images.
- Public or private repositories.
- Used with `docker push` and `docker pull`.

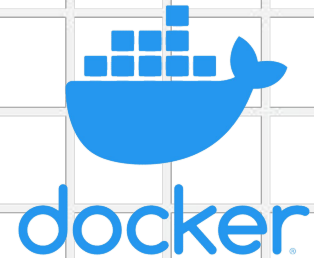


Karan Gupta

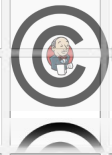


Docker Container - I

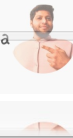
(Run, List, Stop & Remove)



Docker Container Commands



Karan Gupta



- **docker run** – Run a container from an image

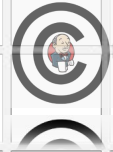
```
docker run [OPTIONS] IMAGE
```

Example:

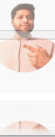
```
docker run -d -p 8080:80 --name myweb nginx
```

Option	Description	Example
-d	Run in detached (background) mode	-d
-p	Map host port to container port	-p 8080:80
--name	Assign a name to the container	--name my_container
-it	Interactive terminal	-it ubuntu bash

Docker Container Commands



Karan Gupta

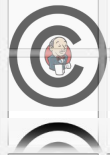


- **docker ps** – List running containers

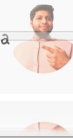
`docker ps [OPTIONS]`

Option	Description	Example
<code>-a</code>	Show all containers (running + stopped)	<code>docker ps -a</code>
<code>-q</code>	Show only container IDs	<code>docker ps -q</code>
<code>--filter</code>	Filter output based on conditions	<code>--filter status=exited</code>

Docker Container Commands



Karan Gupta



- **docker stop** – Gracefully stop a running container

```
docker stop [OPTIONS] CONTAINER
```

Example:

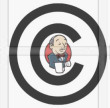
```
docker stop myweb
```

- **docker rm** – Remove one or more containers

```
docker rm [OPTIONS] CONTAINER
```

Example:

```
docker rm myweb
```

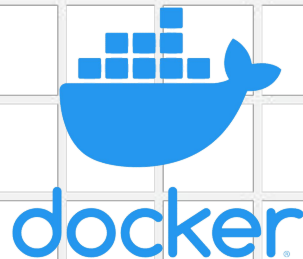


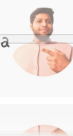
Karan Gupta



Docker Container - II

(Create vs Run vs Start vs Restart)





Command	Purpose	Usage	Example
<code>docker create</code>	Only create a container (does not start it)	Prepares a container for later use	<code>docker create nginx</code>
<code>docker run</code>	Create & start a new container	Runs a container from an image (one step)	<code>docker run -d nginx</code>
<code>docker start</code>	Start an existing (stopped/created) container	Resumes a container that was previously created	<code>docker start <container_id></code>
<code>docker restart</code>	Stop and then start a container again	Used for refreshing a container's process	<code>docker restart <container_id></code>

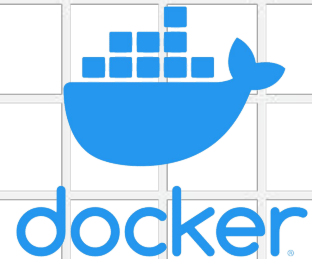


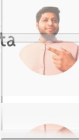
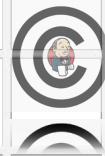
Karan Gupta



Docker Container - III

(Inspect, Rename & Logs)





Command	Purpose	Usage	Example
<code>docker inspect</code>	View detailed info (config, IP, volumes, etc.)	Used for debugging or monitoring	<code>docker inspect myapp</code>
<code>docker rename</code>	Rename an existing container	Changes container name without stopping it	<code>docker rename oldname newname</code>
<code>docker logs</code>	View container logs (stdout/stderr output)	Useful for troubleshooting application issues	<code>docker logs myapp</code>

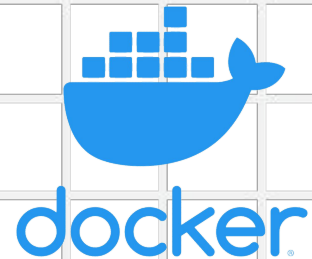


Karan Gupta



Docker Container - IV

(Pause & Unpause)





Command	Purpose	Usage	Example
<code>docker pause</code>	Temporarily suspend all container processes	Freezes the container's execution	<code>docker pause myapp</code>
<code>docker unpause</code>	Resume processes in a paused container	Restarts container's execution	<code>docker unpause myapp</code>

When to Use

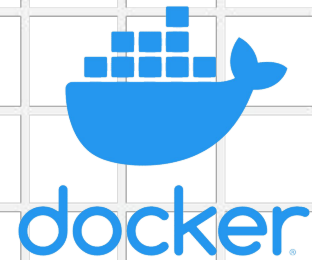
- During maintenance tasks (e.g., disk I/O freeze)
- To temporarily stop execution without shutting down
- To free up system resources temporarily



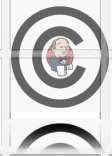
Karan Gupta



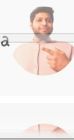
**What is your insight on
Dangling Images and prune
command?**



Dangling Images



Karan Gupta



Dangling images are **unused image layers** that:

- Are not tagged
- Are not referenced by any containers

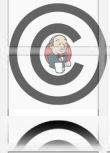
Typically left behind after rebuilding images

Example:

- Rebuilding an image may leave the old, untagged image behind.

Why Remove Them?

- They consume disk space unnecessarily.
- Not useful for deployment or rollback.



Karan Gupta

Docker image prune

- Removes all **dangling images**:

`docker image prune`

Docker prune

- Cleans up:
 - Unused containers

Benefits:

- Frees up disk space.
- Keeps Docker environment clean and efficient.



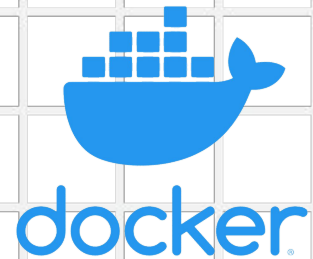
Karan Gupta

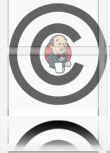


**How we can take backup
and restore a container?**

OR

**Explain export & import
command?**





Docker Export:

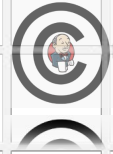
- Used to **export** the filesystem of a container (not the image).
- Creates a **.tar** archive of the container's file system.
- Common use: Moving containers between environments or archiving.

```
docker export <container_id> > container.tar
```

Docker Import

- Used to **create an image** from a container archive.
- Reconstructs the container as a new image.

```
docker import container.tar <image_name>
```



Karan Gupta



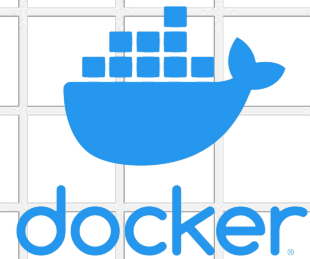
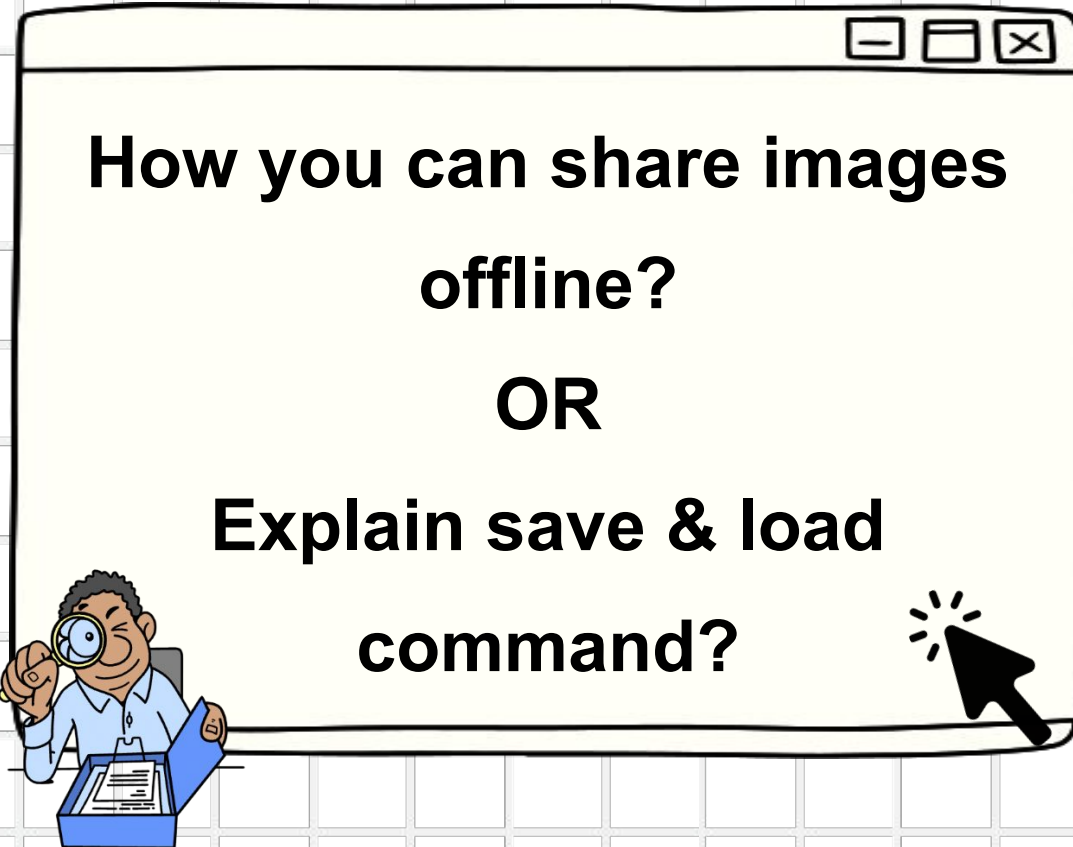
Use Cases:

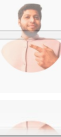
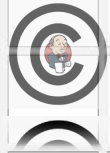
- Backup and restore containers.
- Migrate containers between hosts.
- Share containers without using Docker Hub.





Karan Gupta





Docker Save:

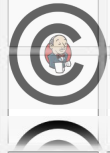
- Used to **save a Docker image** as a **.tar** archive.
- Ideal for transferring images between systems without using Docker Hub.

```
docker save -o image.tar <image_name>
```

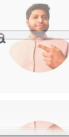
Docker Load:

- Used to **load a Docker image** from a **.tar** archive.
- Reconstructs the saved image on a new Docker host.

```
docker load -i image.tar
```



Karan Gupta



Use Cases:

- Sharing Docker images offline.
- Version-controlled backups of Docker images.
- Moving images between air-gapped environments.

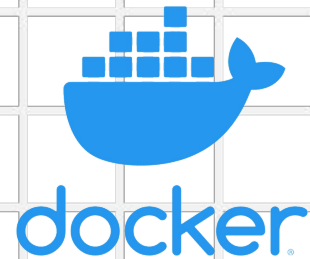




Karan Gupta



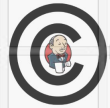
**Differentiate between
save/load & export/import.**



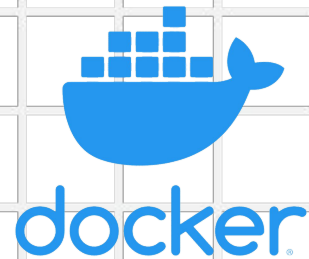
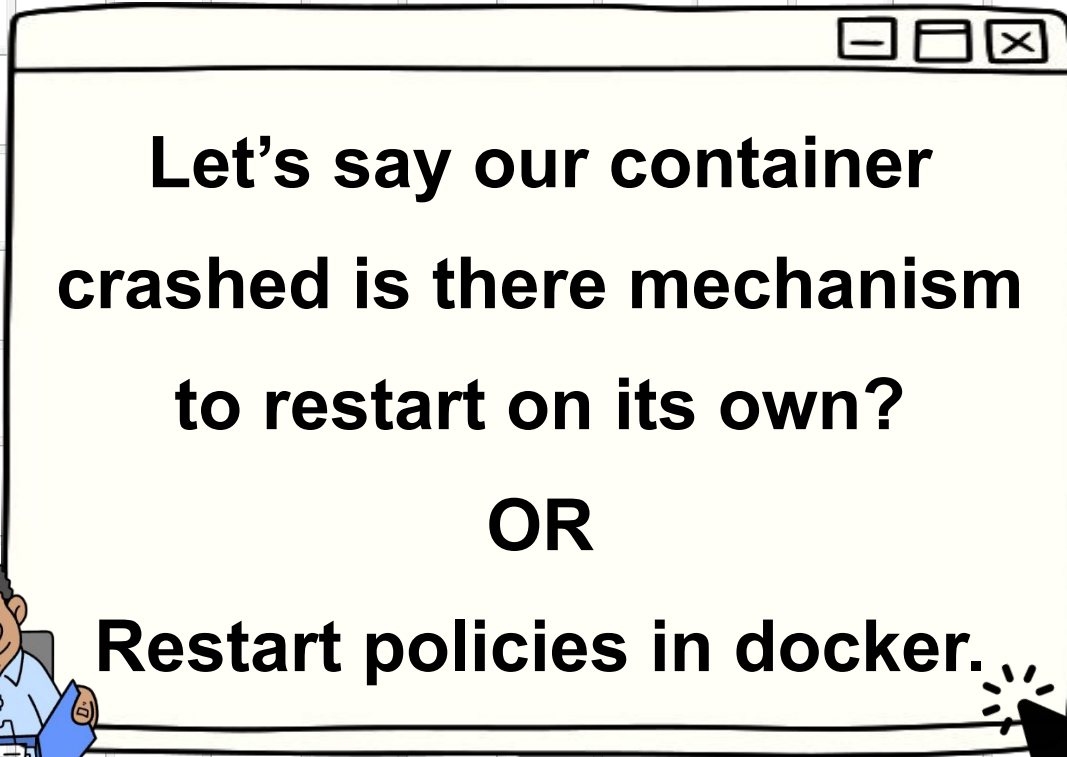


Feature	Save / Load	Export / Import
Target	Docker Image	Running Container
Metadata	Preserved (tags, layers)	Not preserved
Output	Image archive (.tar)	Filesystem snapshot (.tar)
Use Case	Transfer full image history	Snapshot for migration

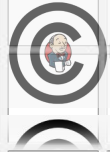
- Use **export/import** when container state matters, and **save/load** when preserving image history is crucial.
- **docker export/import** deals with containers.
- **docker save/load** deals with images.



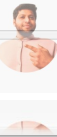
Karan Gupta



Restart Policies in Docker



Karan Gupta



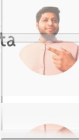
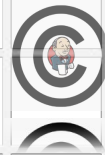
What Are Restart Policies?

- Docker restart policies define **whether and when to restart a container** automatically after it exits or fails.

Why Use Them?

- Improve container availability.
- Automatically recover from crashes or reboots.

```
docker run --restart=<policy> <image>
```



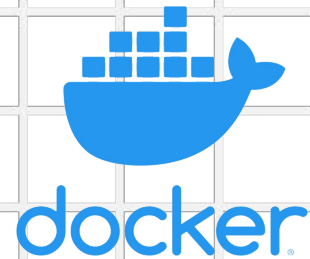
Restart Policy	Description	Use Case Example
<code>no</code> (<i>default</i>)	Do not restart the container automatically.	One-time jobs or short-lived containers.
<code>always</code>	Always restart the container if it stops, including after Docker daemon restart.	Web servers, background workers, essential services.
<code>on-failure[:n]</code>	Restart only when the container exits with a non-zero status. Optional retry <code>n</code> .	Retry error-prone tasks or flaky apps (e.g., scripts).
<code>unless-stopped</code>	Same as <code>always</code> , except when stopped manually by the user.	Services that should stay up unless explicitly stopped.



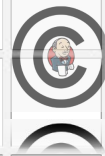
Karan Gupta



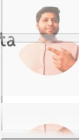
**What are different ways to
connect to a container?
OR
Explain about docker
networking.**



Docker Networking



Karan Gupta



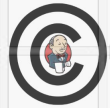
- Enables containers to **communicate** with each other, the host, and external networks.
- Docker creates a **default network** for containers, but also allows custom network setups.

Benefits:

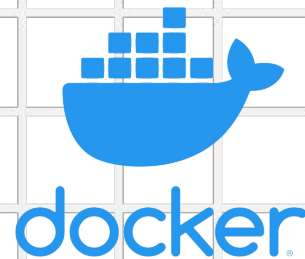
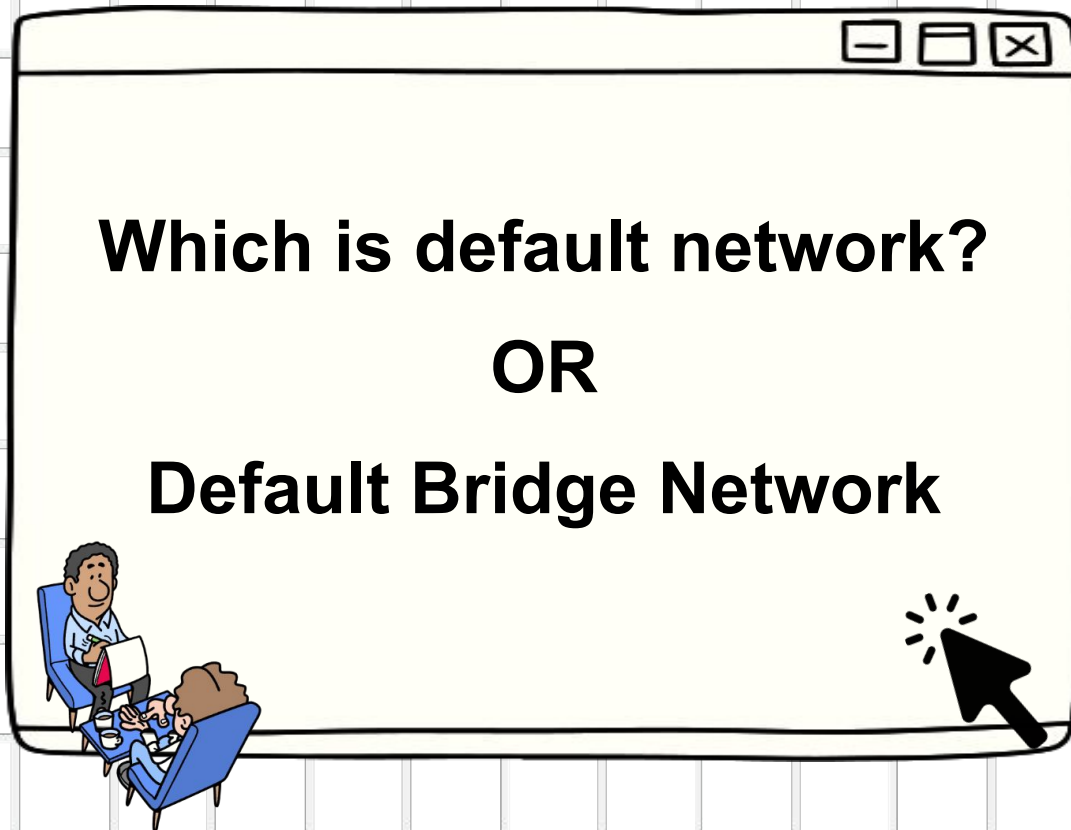
- Flexible container-to-container communication.
- Network-level isolation and security.
- Simplified service discovery within Docker networks.

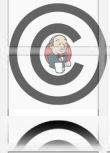


Network Type	Description	Typical Use Case
bridge	Default network for containers on a single host; uses NAT	Isolated communication between containers on same host
host	Container shares the host's network stack	Performance-sensitive apps like logging or DNS
none	Container has no network access	Security testing, complete isolation
overlay	Enables container communication across multiple hosts	Distributed microservices in Docker Swarm
macvlan	Assigns MAC address to each container; Layer 2 network	Direct LAN access, legacy systems requiring static MACs
ipvlan	Assigns IPs at Layer 3; avoids MAC limitations	Scalable IP-based routing in large containerized networks

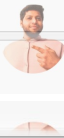


Karan Gupta





Karan Gupta

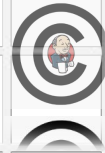


Default – Bridge Network

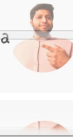
- Docker creates a **default bridge network** named **bridge** when installed.
- Used when no custom network is specified in **docker run**.

Characteristics:

- Isolates containers from the host.
- Enables **container-to-container** communication using IP addresses.
- DNS-based container name resolution **not supported** by default **bridge**.



Karan Gupta

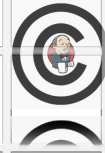


Behavior:

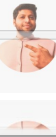
- Containers get a private IP in the 172.x.x.x range.
- Can communicate **with the outside world** but **not with each other by name** (only by IP).

Use Case:

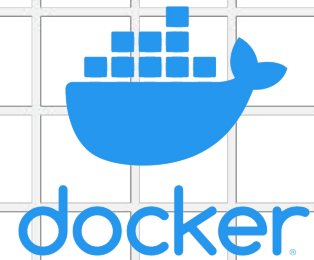
- Quick tests or isolated containers.
- Simple apps without the need for inter-container communication.

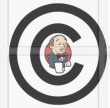


Karan Gupta



Creating Dockerfile with ifconfig and ping command

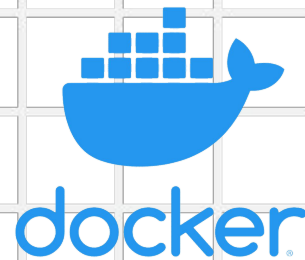


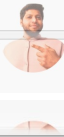
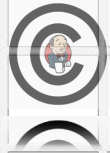


Karan Gupta



What is Host Network?



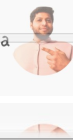
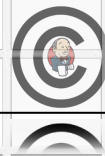


Host Network

- The **host network** mode makes the container **share the host's network stack** directly.
- The container does **not get its own IP address** — it uses the **host's IP and ports**.

Characteristics:

- No network isolation between container and host.
- No port mapping (**-p** is ignored).
- Better **performance and lower latency**



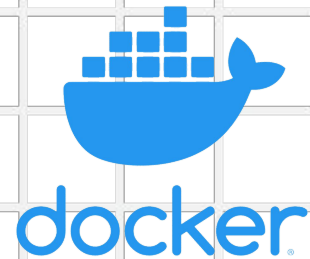
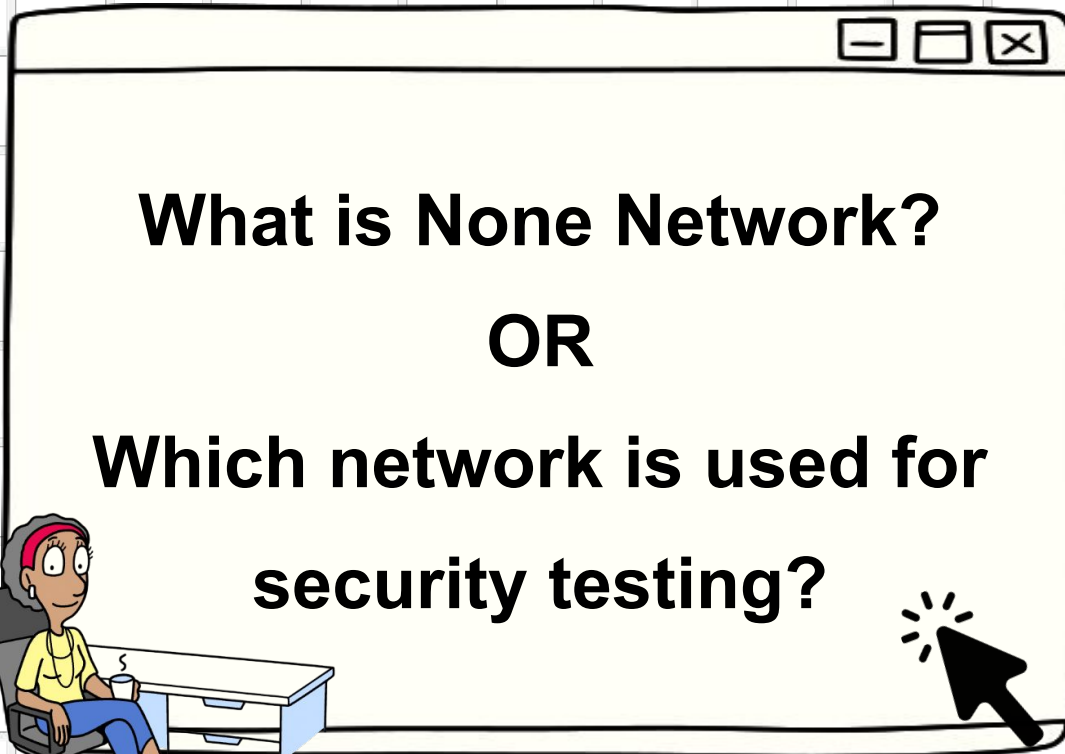
Use Case	Description
High-performance networking	Apps like DNS, syslog, or load balancers
Port binding simplification	Direct port access without mapping
Low-latency requirements	Apps that benefit from bypassing Docker's virtual network layer

Usage:

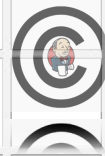
```
docker run --network host nginx
```



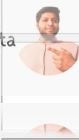
Karan Gupta



None Network



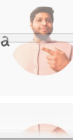
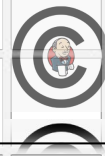
Karan Gupta



- The **none** network driver **disables all networking** for the container.
- The container gets its **own network namespace**, but **no external connectivity** (only loopback **127.0.0.1**).

Characteristics:

- No default gateway
- No DNS resolution
- No internet access
- Used mainly for **testing isolation and security**



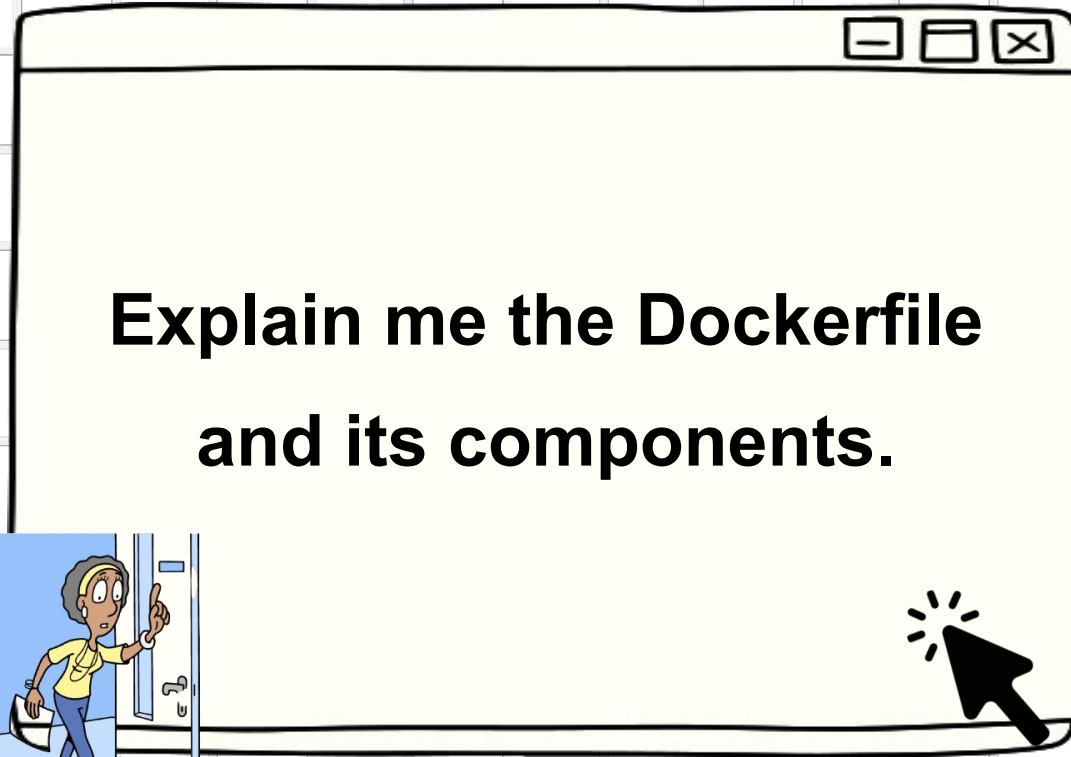
Scenario	Purpose
Security testing	Ensure app doesn't access external resources
Dependency-free script testing	Run self-contained scripts or binaries
Debugging container startup	Check init behavior without interference

Usage:

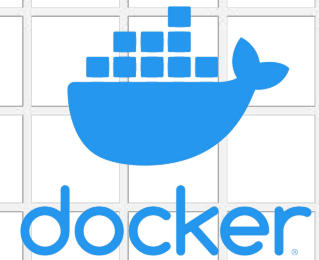
```
docker run --network none ubuntu
```



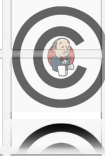
Karan Gupta



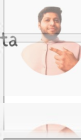
**Explain me the Dockerfile
and its components.**



Dockerfile



Karan Gupta



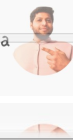
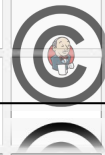
- A **Dockerfile** is a **text file** that contains a set of instructions to build a Docker image automatically.

Purpose:

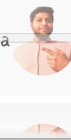
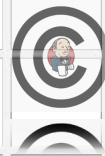
- Automate image creation
- Ensure consistency
- Version control for container configurations

Benefits:

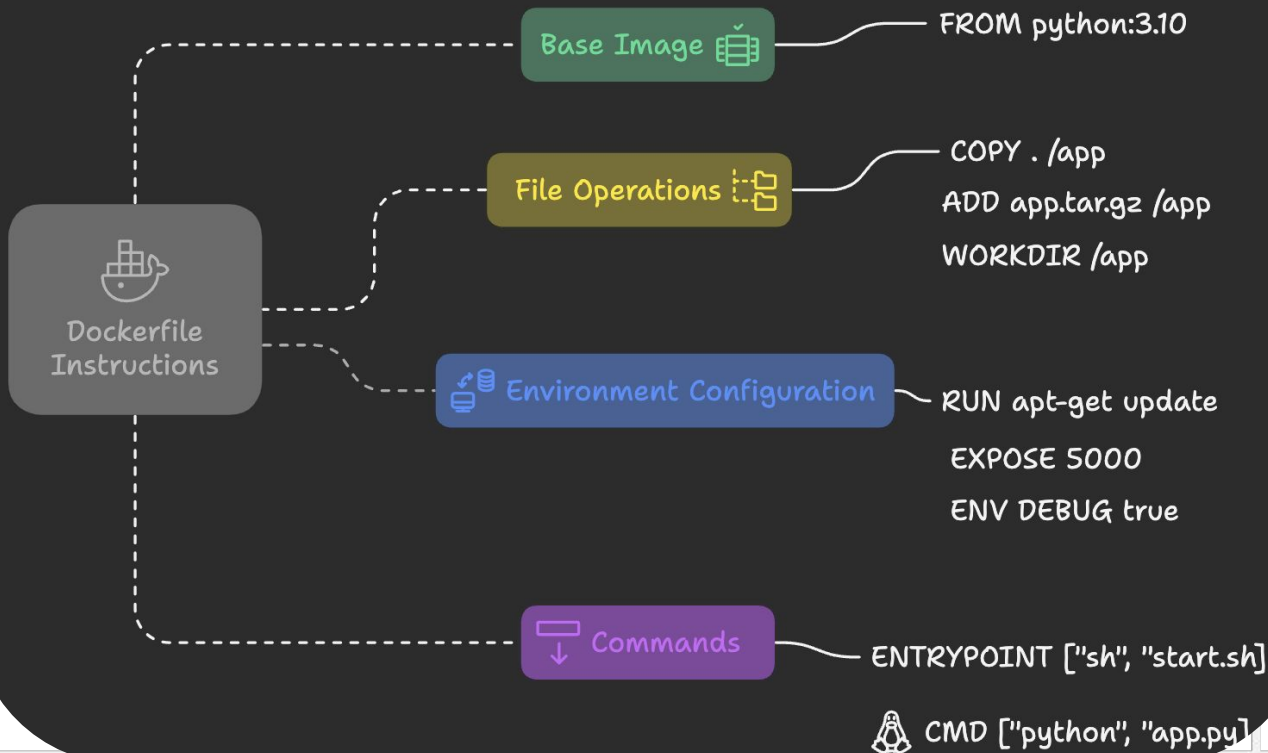
- Reproducible builds
- Easy to share and deploy
- Simplifies environment setup



Instruction	Description	Example
FROM	Base image to build from	FROM python:3.10
RUN	Execute commands in shell	RUN apt-get update
COPY	Copy files into image	COPY ./app
ADD	Copy files (with auto-extract)	ADD app.tar.gz /app
WORKDIR	Set working directory	WORKDIR /app
CMD	Default command to run	CMD ["python", "app.py"]
EXPOSE	Document container port	EXPOSE 5000
ENV	Set environment variable	ENV DEBUG true
ENTRYPOINT	Configure container as executable	ENTRYPOINT ["sh", "start.sh"]



Dockerfile Instructions and Their Functions





Karan Gupta



sample

Users > karangupta > Downloads > star > aws trainings > sample > ...

```
1  # Start from a base image
2  FROM python:3.9
3
4  # Set the working directory
5  WORKDIR /app
6
7  # Copy application code
8  COPY . .
9
10 # Install dependencies
11 RUN pip install -r requirements.txt
12
13 # Define the command to run the app
14 CMD ["python", "app.py"]
15
```

`docker build -t myapp-image .`



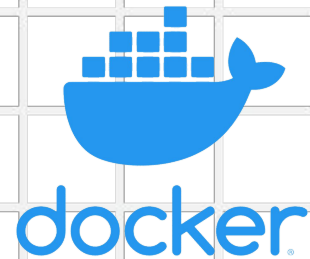
Karan Gupta



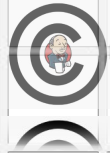
**How to access the container
externally?**

OR

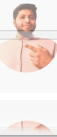
What is port mapping?



Port Mapping



Karan Gupta



- Port mapping connects a port on the **host machine** to a port inside the **Docker container**, enabling external access to containerized services.

```
docker run -p <host_port>:<container_port> <image>
```

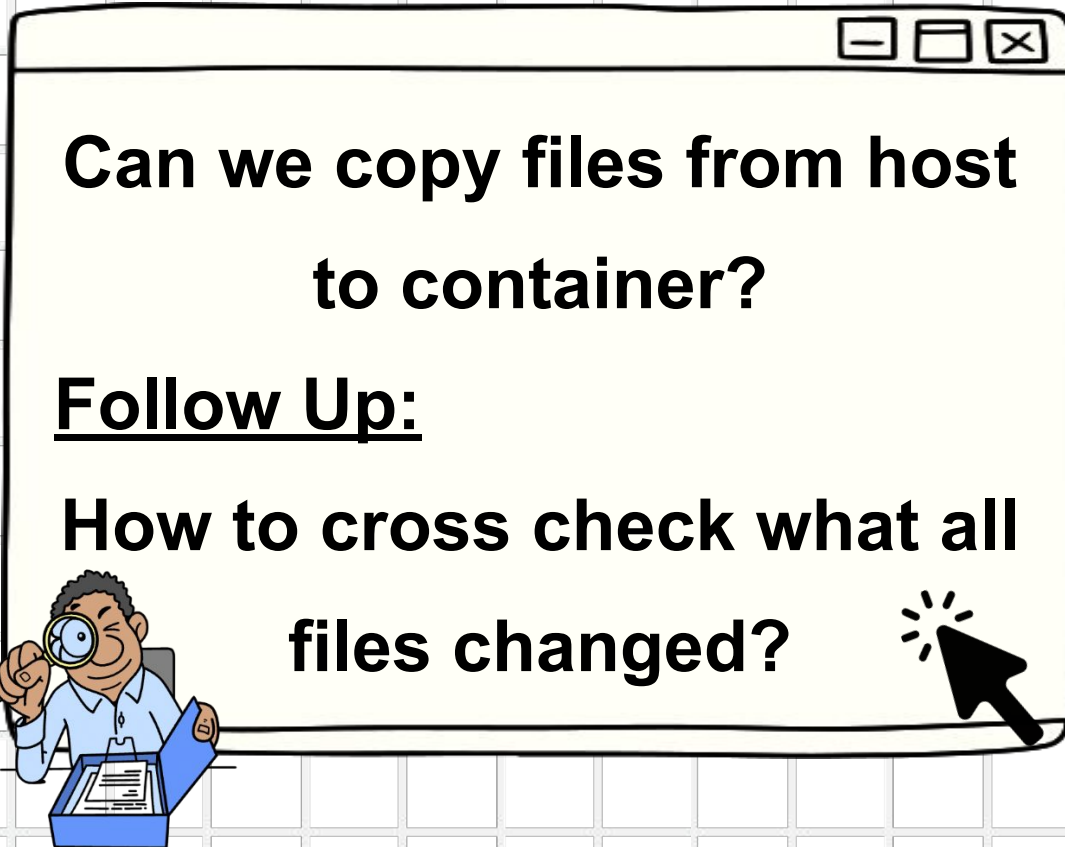
Example:

```
docker run -d -p 8080:80 nginx
```

- Maps host port **8080** to container port **80**



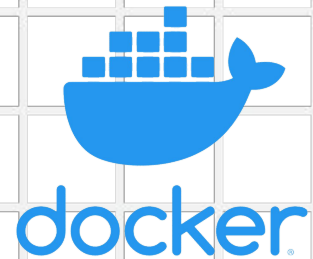
Karan Gupta



**Can we copy files from host
to container?**

Follow Up:

**How to cross check what all
files changed?**





Docker Copy:

- The `docker cp` command copies **files or folders between a container and the local filesystem.**

```
docker cp <container>:/path/in/container /host/path
```

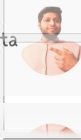
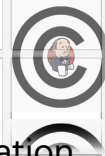
```
docker cp /host/path <container>:/path/in/container
```

Use Cases:

- Retrieve logs or config files
- Inject files into a running container
- Backup/restore container content

```
docker cp mycontainer:/var/log/nginx/access.log ./access.log
```

```
docker cp ./index.html mycontainer:/usr/share/nginx/html/
```



Docker Diff:

- The `docker diff` command **shows changes to the container's filesystem** since its creation.

```
docker diff <container>
```

Output:

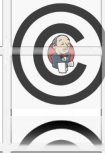
```
C /app
```

```
A /app/newfile.txt
```

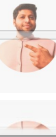
```
D /tmp/tempfile
```

Use Cases:

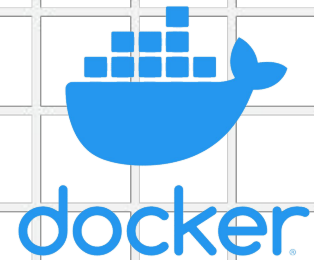
- Audit what has changed inside a container
- Debug unexpected file modifications
- Track runtime changes in development containers



Karan Gupta



Docker User Defined Network

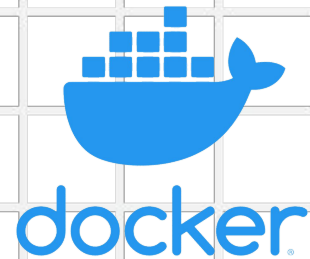


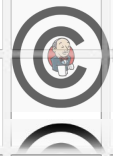


Karan Gupta

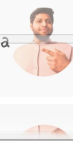


What are the benefits of Docker and alternatives of it?

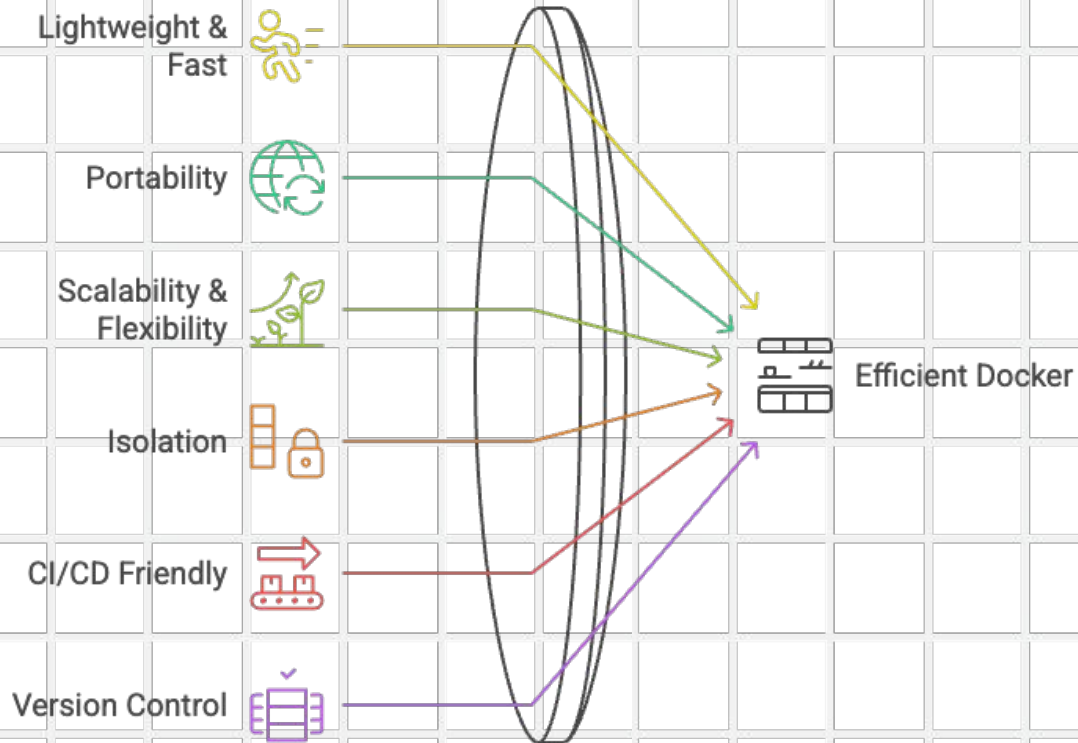


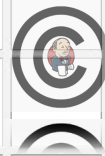


Karan Gupta

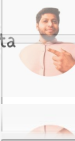


The Power of Docker

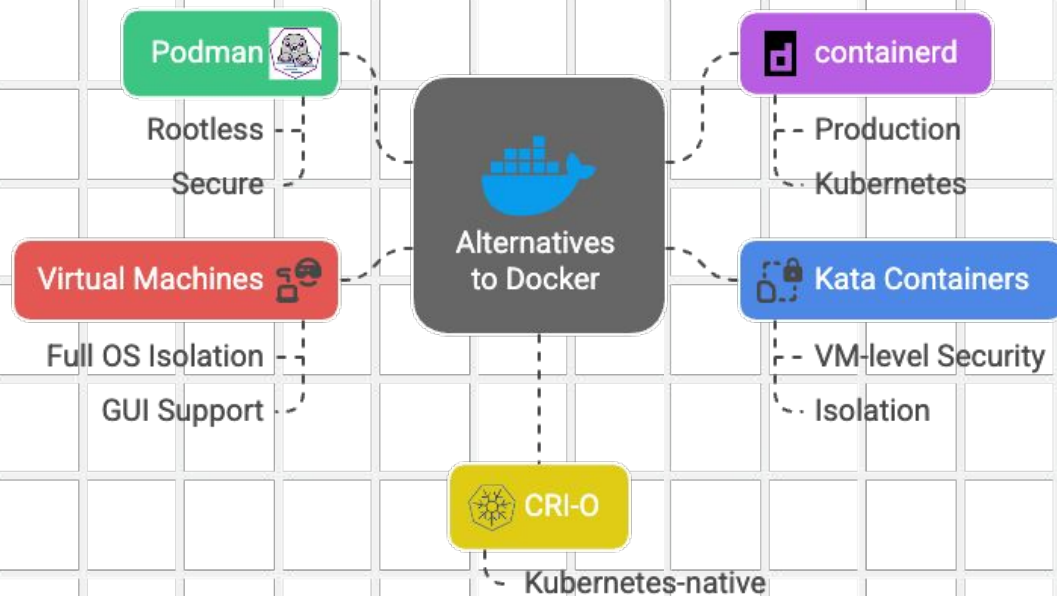


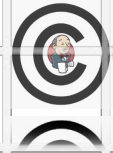


Karan Gupta

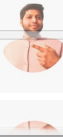


Alternatives to Docker for Containerization

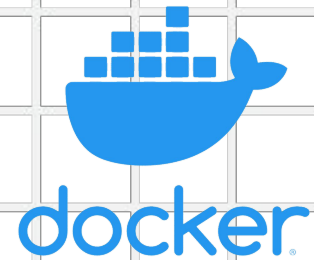




Karan Gupta

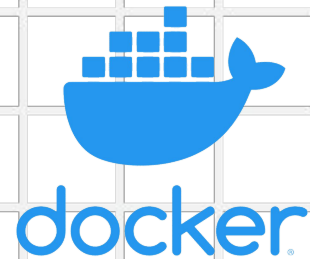
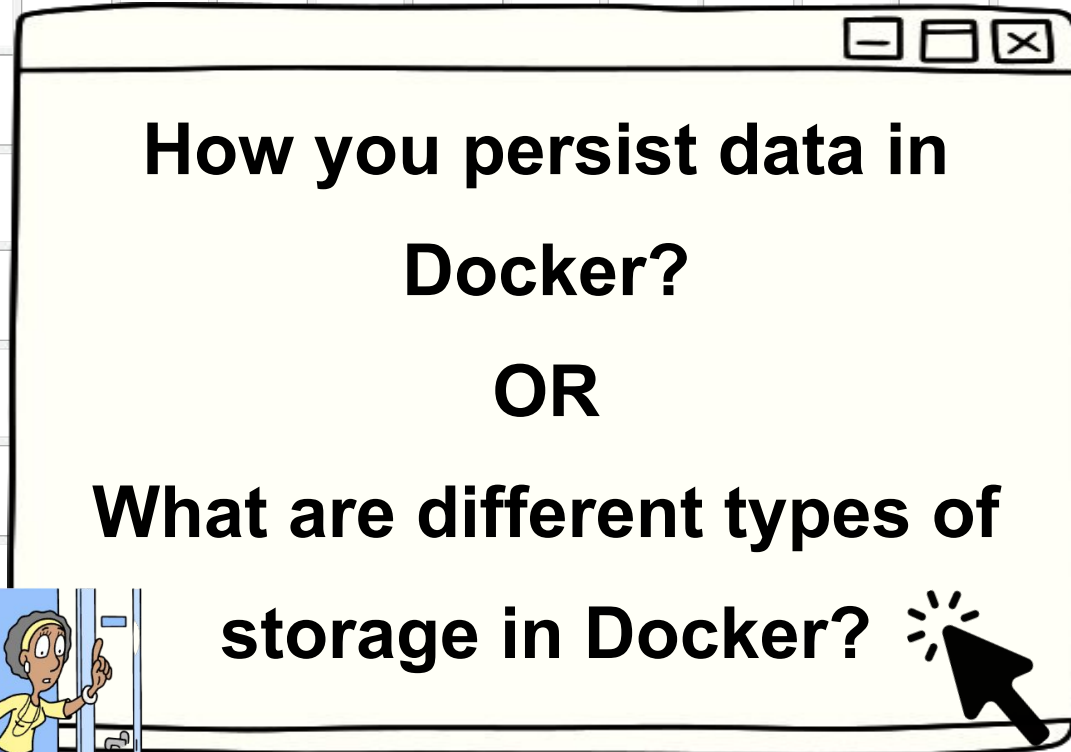


Docker Help & Manual

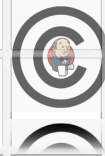




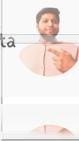
Karan Gupta



Docker Storage



Karan Gupta



- By default, data created in containers is **ephemeral**—deleted when the container is removed.

Docker volumes **persist data** outside the container's writable layer.

Docker Storage Types

Volumes

Managed by Docker,
stored in
`/var/lib/docker/volumes`.



Bind Mounts

Host directory mapped
directly to container.

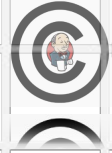


tmpfs

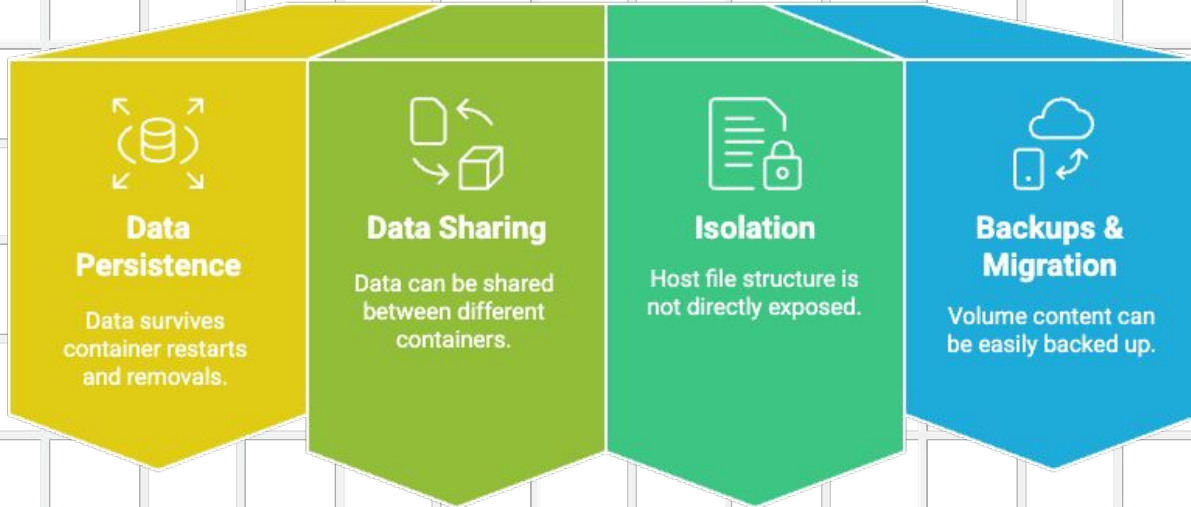
Temporary storage in
memory only.



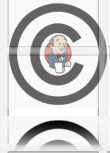
Docker Volume Benefits



Karan Gupta



Scenario	Reason for Using Volumes
Database containers	Persist DB files on disk
App logs or uploads	Store user files or logs
Config sharing	Share config between services



Karan Gupta

```
docker volume ls          # List volumes
docker volume inspect NAME # View details
docker volume rm NAME     # Remove volume
```

Volume

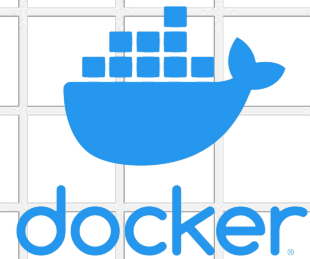
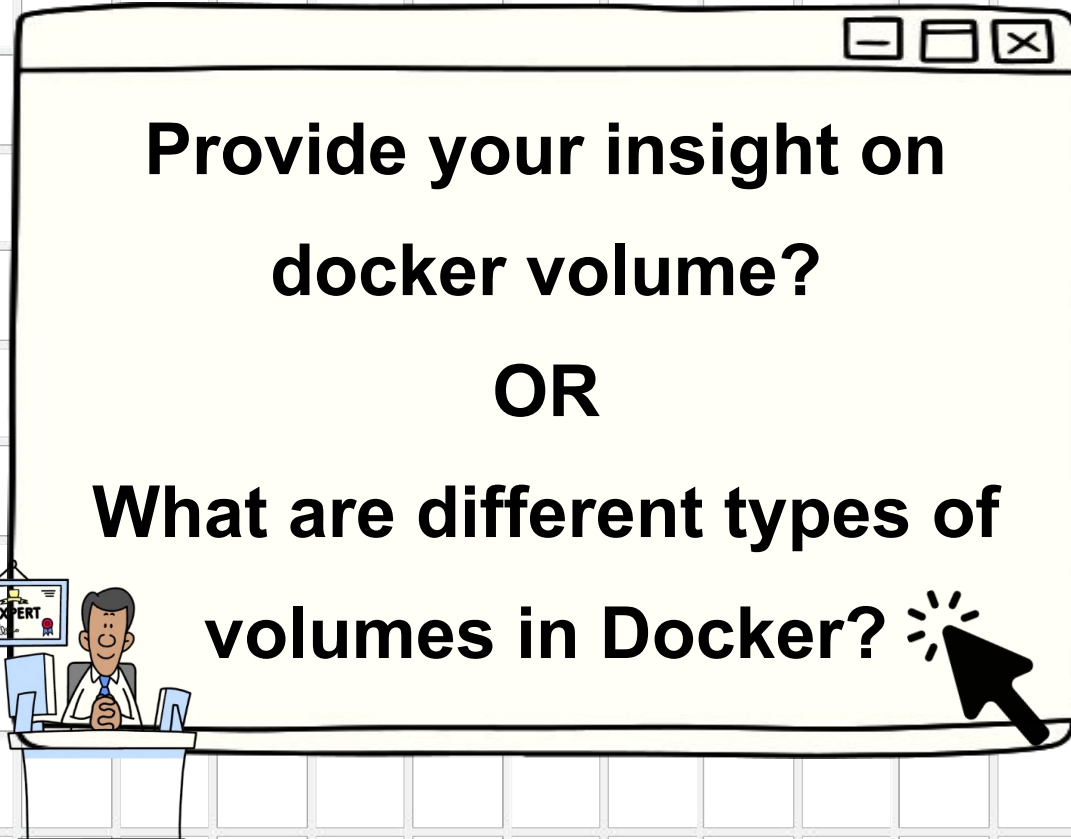
```
docker volume create myvol
docker run -v myvol:/data myimage
```

Bind Mount

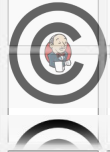
```
docker run -v /host/data:/data myimage
```




Karan Gupta



Docker Volume

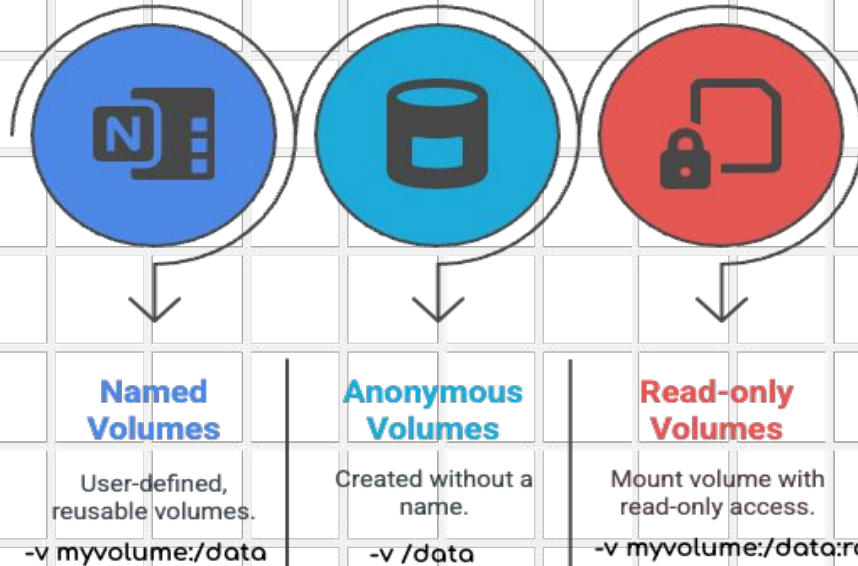


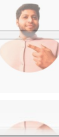
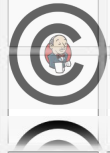
Karan Gupta



- Volumes are Docker-managed storage that lives outside the container lifecycle.

Docker Volume Types





Syntax:

```
docker volume create myvolume  
docker run -v myvolume:/app/data nginx  
docker run -v myvolume:/app/data:ro nginx  
docker run -v /app/data nginx # anonymous
```

Notes:

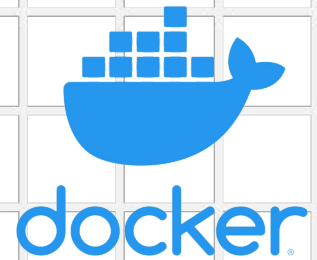
- **Named volumes** are easy to manage and inspect.
- **Anonymous volumes** can accumulate and require cleanup.
- **Read-only volumes** enhance security but prevent writes.



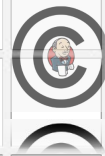
Karan Gupta



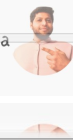
What is Bind Mount Storage?



Bind Mount



Karan Gupta



- Bind mounts map a **specific path on the host machine** directly into a container.

Bind Mount Features

Direct Access

Container directly accesses the host filesystem.



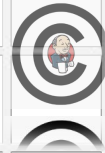
Path Requirement

Requires specifying the full path on the host.

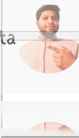


Instant Reflection

Changes to host files instantly appear in container.



Karan Gupta

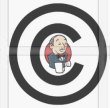


Syntax:

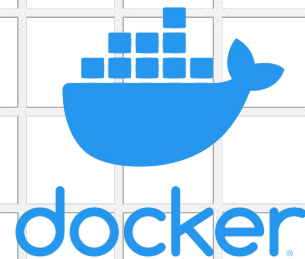
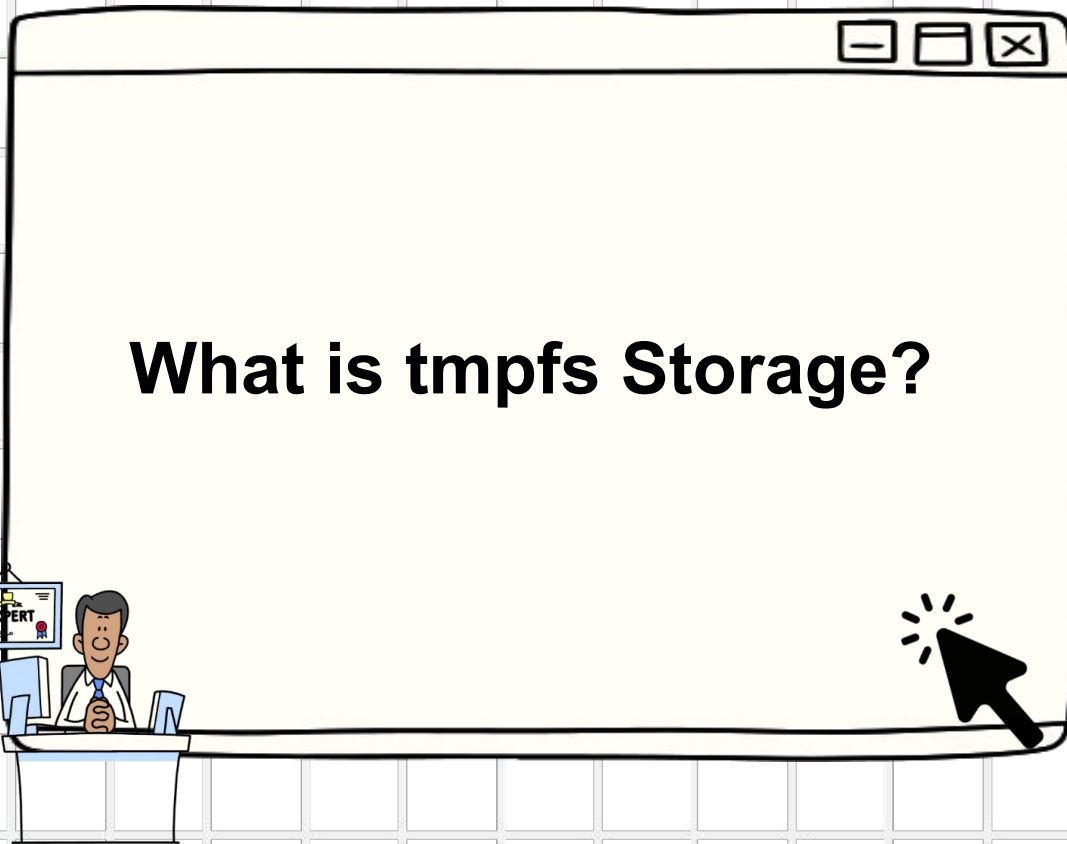
```
docker run -v /host/data:/container/data nginx
```

Use Case:

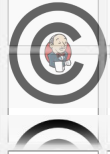
- Sharing source code for live development
- Accessing host logs or config files
- Mounting custom SSL certificates or secrets



Karan Gupta



tmpfs



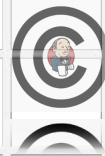
Karan Gupta



- A **tmpfs** mount is a **temporary filesystem in memory** (RAM), not written to disk.

Tmpfs Characteristics





Syntax:

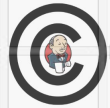
```
docker run --tmpfs /app/cache:rw nginx
```

Use Case:

- Storing sensitive information (e.g., tokens)
- Speed-critical temporary cache
- Session data for web apps

Notes:

- Only supported on **Linux** hosts.



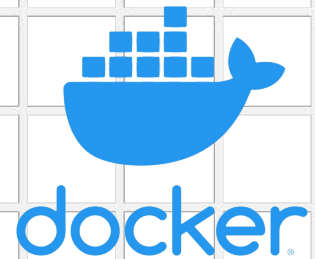
Karan Gupta



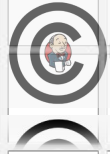
**Where you can store docker
images?**

OR

What is DockerHub?



DockerHub



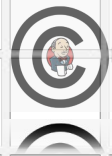
Karan Gupta



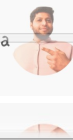
- Docker Hub is a **cloud-based registry** where developers can **store, share, and distribute Docker images**.

Key Features:

- **Official Images:** Prebuilt and verified by Docker (e.g., **nginx**, **redis**)
- **User Images:** Host your own public or private images
- **Automated Builds:** Link GitHub/GitLab → build & push images on code change
- **Access Control:** Fine-grained permission for private repositories



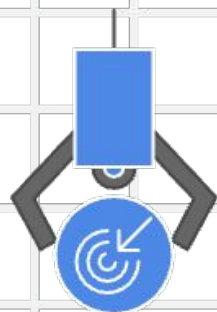
Karan Gupta



Basic Commands:

- docker login
- docker pull nginx
- docker push myusername/myimage:latest

DockerHub Features



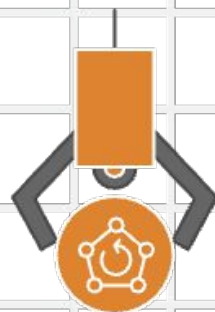
CI/CD Integration

Automate builds and
deployments



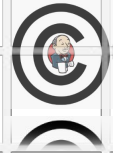
Image Distribution

Share base or app
images across
environments

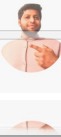


Backup/Versioning

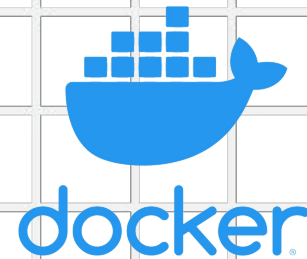
Track versions of
images using tags

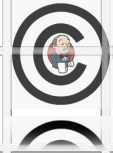


Karan Gupta



Pulling Image from DockerHub

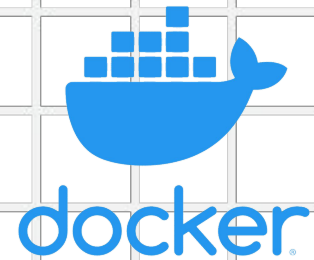


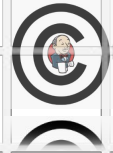


Karan Gupta



Build Project Locally - NodeJs

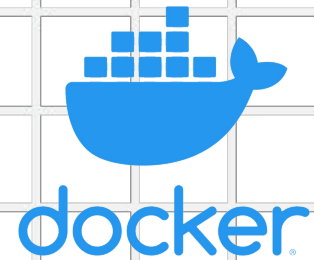




Karan Gupta

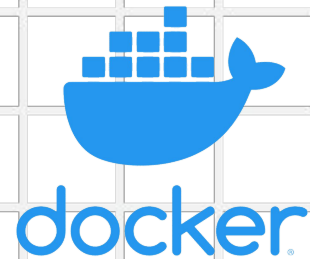
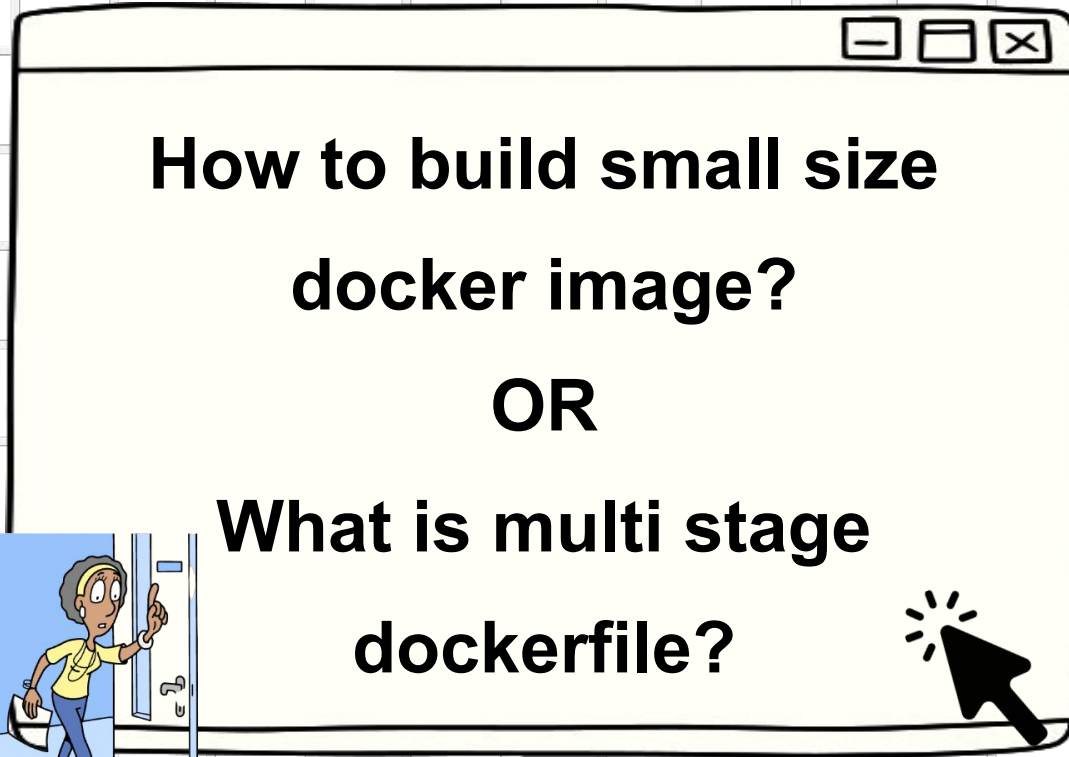


Multi-Stage Dockerfile

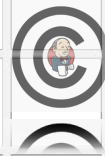




Karan Gupta



Multi-Stage Dockerfile



Karan Gupta

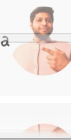
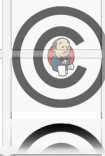
- Multi-stage builds use **multiple FROM instructions** in a Dockerfile to **separate build and runtime environments**.

Purpose:

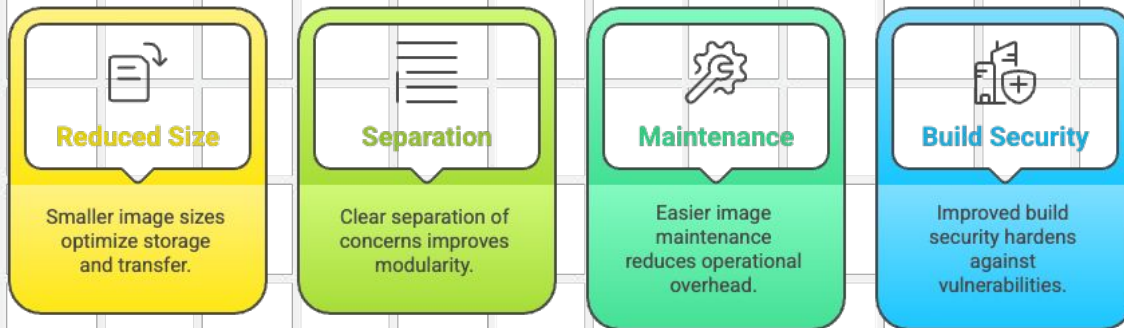
- Keep final image **lightweight and secure**
- Avoid shipping unnecessary build tools and files

How It Works:

- **Stage 1:** Build/compile your app (with full dev dependencies)
- **Stage 2+:** Copy only what's needed into minimal base image

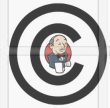


Multi-Stage Dockerfile Benefits

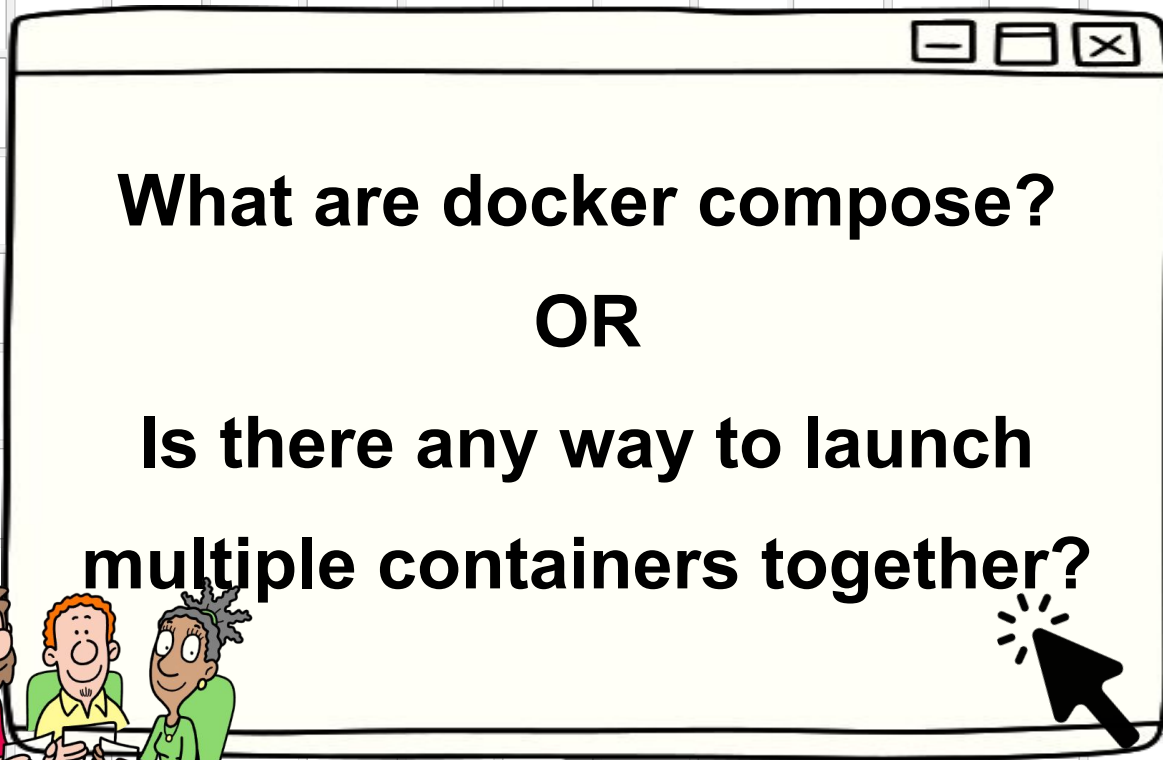


Common Use Cases:

- Node.js apps (separating build from runtime)
- Java apps (build with Maven, run with JRE)
- Static site generators (build → serve with Nginx)
- Go/compiled languages (build binary → run in minimal base)



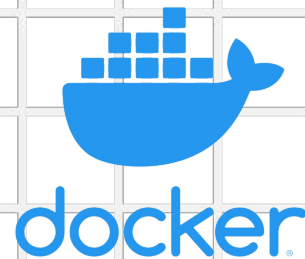
Karan Gupta

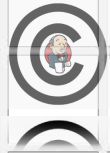


What are docker compose?

OR

**Is there any way to launch
multiple containers together?**





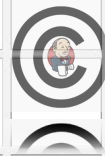
Docker Compose

- A tool for defining and running **multi-container Docker applications** using a YAML file.
-  **Configuration File:** `docker-compose.yml`

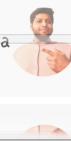
Defines services, networks, and volumes for your application.

Key Features:

- Define and manage **multi-service apps**
- Use `docker compose up` to build, start, and run containers
- Supports scaling and service dependencies



Karan Gupta

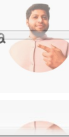
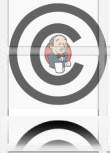


Benefits of Docker Compose

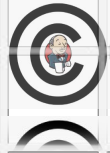


Basic Commands:

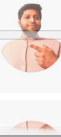
- Docker compose build
- Docker compose up
- Docker compose down
- Docker compose ps



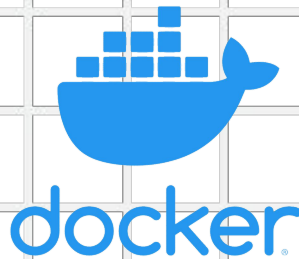
```
1  version: '3.8'
2  √ services:
3  √   web:
4      build: ./app
5  √   ports:
6      |   - "3000:3000"
7  √   db:
8      image: mongo
9  √   ports:
10      |   - "27017:27017"
```

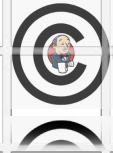


Karan Gupta



Docker Compose Installation



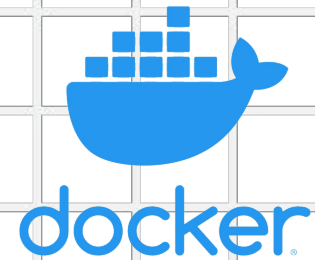


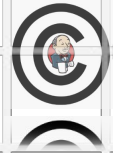
Karan Gupta



Docker Compose

HandsOn-1



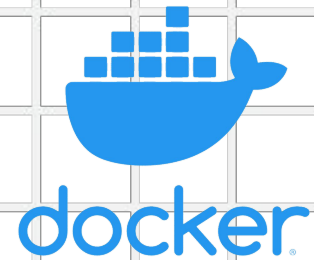


Karan Gupta



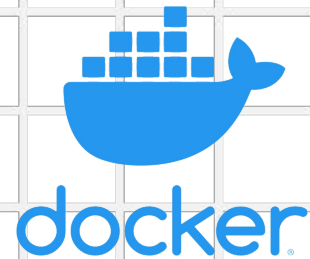
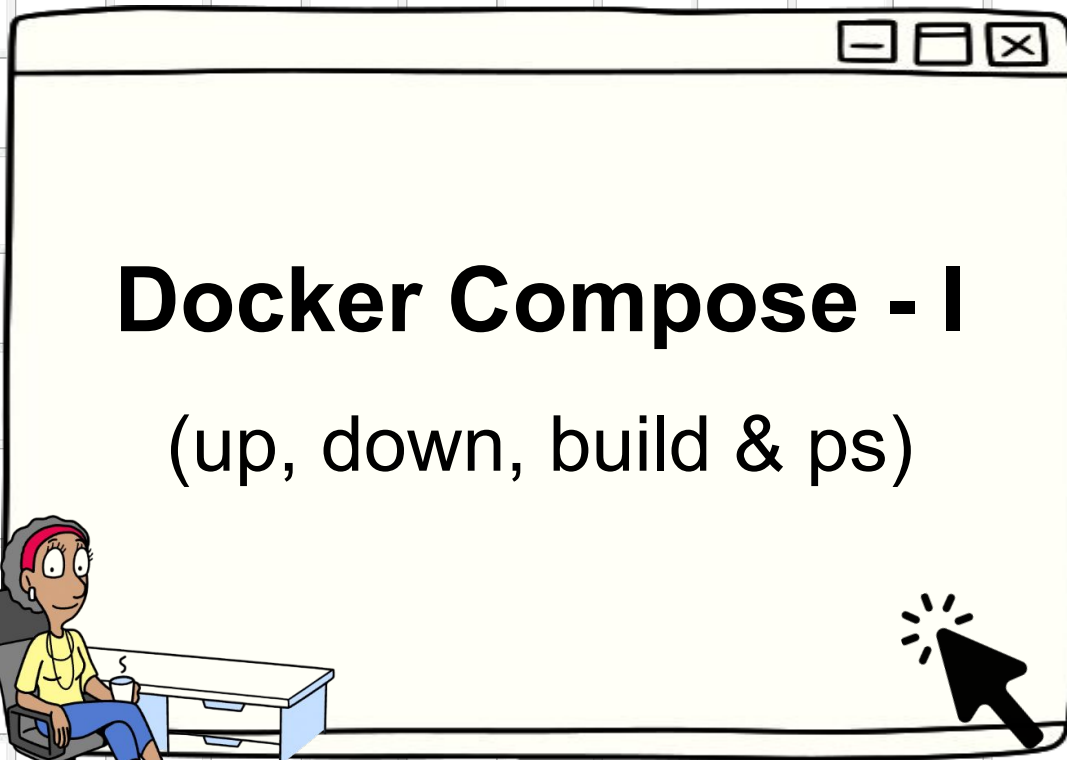
Docker Compose

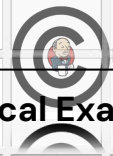
HandsOn-II



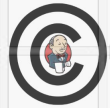


Karan Gupta

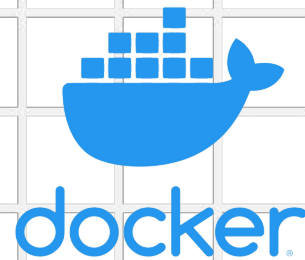
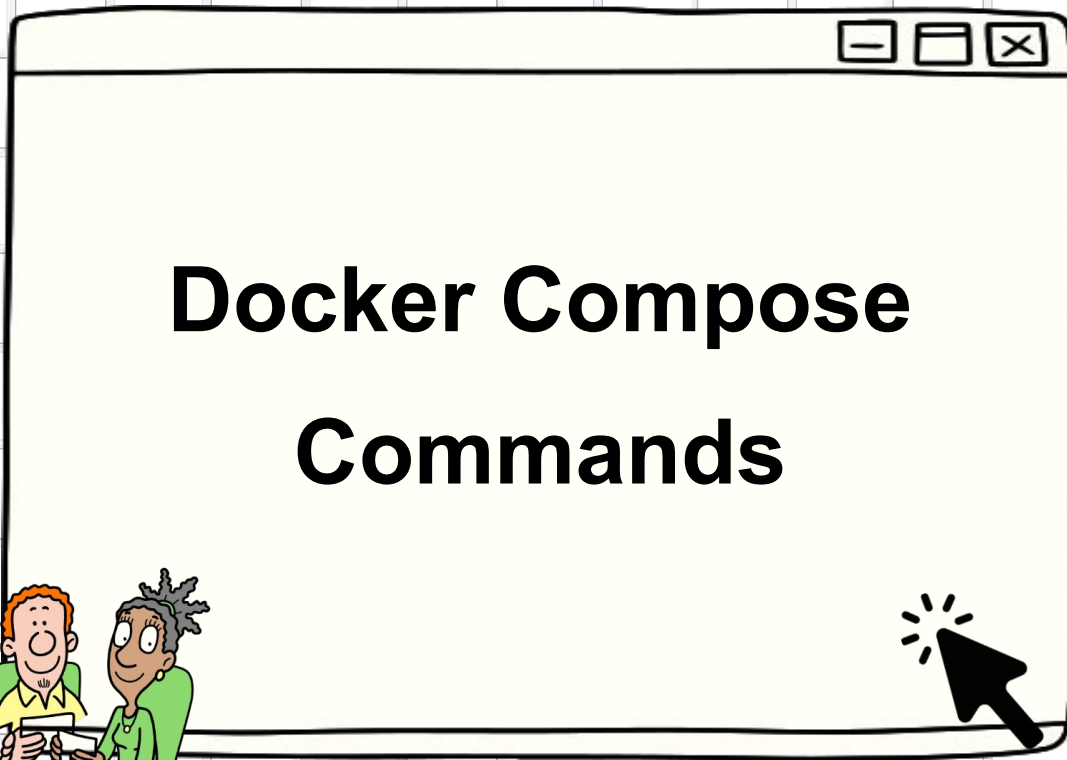




Command	Purpose	Common Flags	Practical Example
<code>docker compose up</code>	Builds (if needed), starts, and runs all services in the background or foreground	<code>-d</code> : Detached mode <code>--build</code> : Rebuild images	<code>docker compose up -d --build</code>
<code>docker compose down</code>	Stops and removes containers, networks, and volumes defined in the file	<code>--volumes</code> : Remove named volumes	<code>docker compose down --volumes</code>
<code>docker compose build</code>	Builds or rebuilds the services defined in the Compose file	<code>--no-cache</code> : Ignore cache	<code>docker compose build</code>
<code>docker compose ps</code>	Lists containers started by Docker Compose	<code>-a</code> : Show all containers	<code>docker compose ps</code>

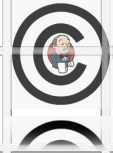


Karan Gupta





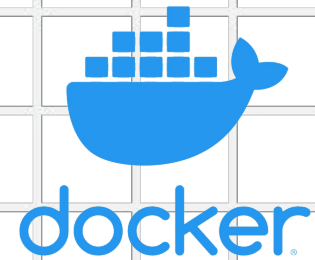
Docker Command	Docker Compose Equivalent	Usage / Description
<code>docker build .</code>	<code>docker compose build</code>	Build images from Dockerfile manually or from Compose services
<code>docker run <image></code>	<code>docker compose up</code>	Run a container manually or start all services from docker-compose.yml
<code>docker start <container></code>	<code>docker compose start</code>	Start stopped containers (already created)
<code>docker stop <container></code>	<code>docker compose stop</code>	Stop running containers
<code>docker ps</code>	<code>docker compose ps</code>	List running containers; shows Compose-specific service status

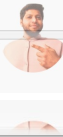
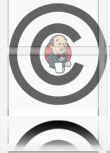


Karan Gupta

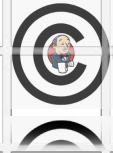


Latest Format of docker compose





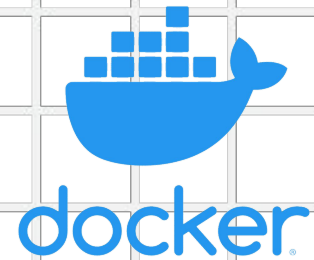
```
1   name: project
2
3  ∨ services:
4  ∨   web:
5      build: ./app
6  ∨   ports:
7      - "3000:3000"
8  ∨   db:
9      image: mongo
10  ∨   ports:
11      - "27017:27017"
```

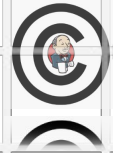


Karan Gupta

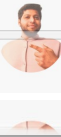


Docker Compose HandsOn

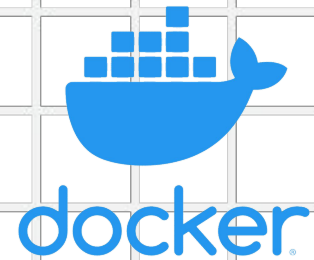


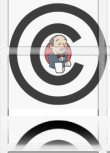


Karan Gupta

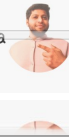


Docker Compose File **(Service Dockerfiles path)**

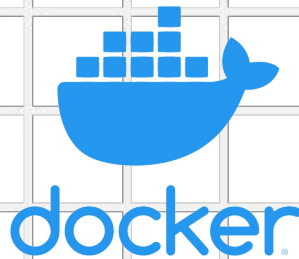


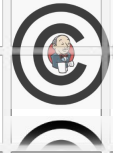


Karan Gupta



Docker Compose File (Volumes)

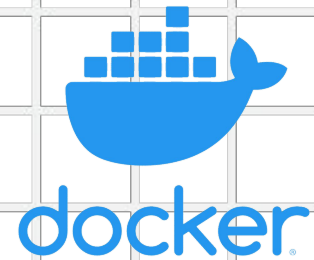


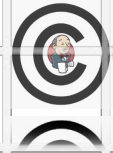


Karan Gupta



Docker Compose File (Wordpress + MySQL)

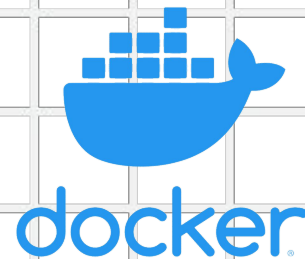


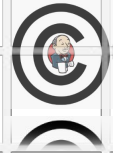


Karan Gupta



Docker Monitoring (cAdvisor)



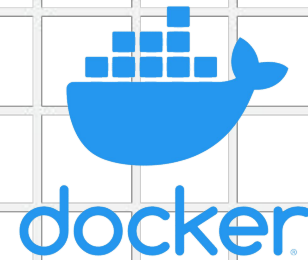


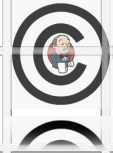
Karan Gupta



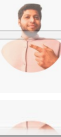
Docker Limits

(Memory - Hard & Soft)

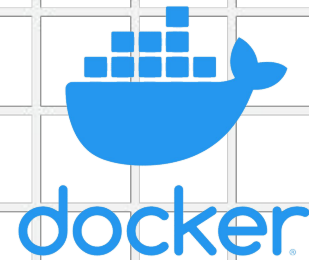




Karan Gupta

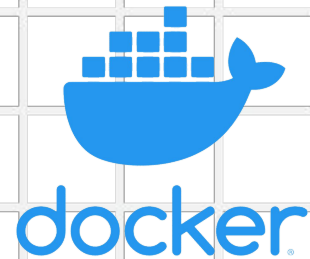
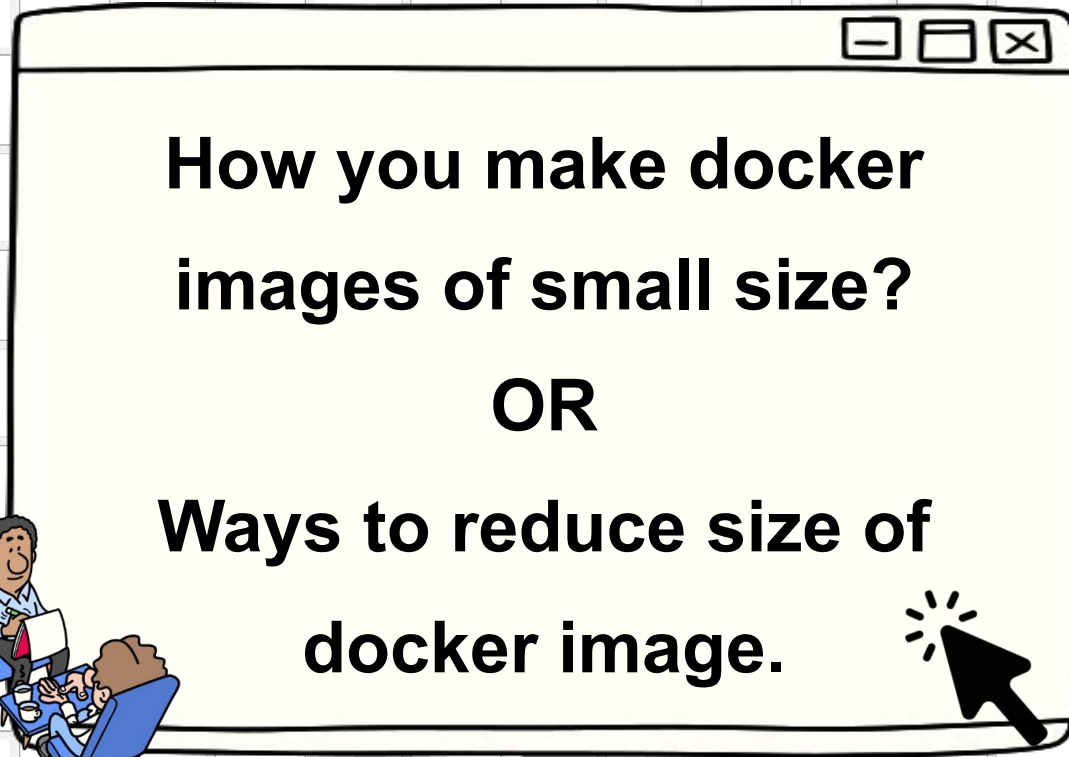


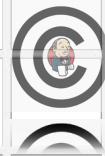
Docker Limits (CPU - Hard & Soft)



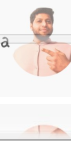


Karan Gupta





Karan Gupta



How to reduce the size of a Docker image?



Use Minimal Base Image

Reduces initial image size by using smaller base images like Alpine.



Use Multi-Stage Builds

Separates build and runtime stages, copying only necessary files.



Clean Up Packages

Removes unnecessary files after installation to save space.



Avoid Unused Packages

Prevents installation of unnecessary tools and utilities.

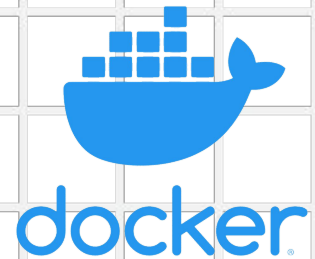
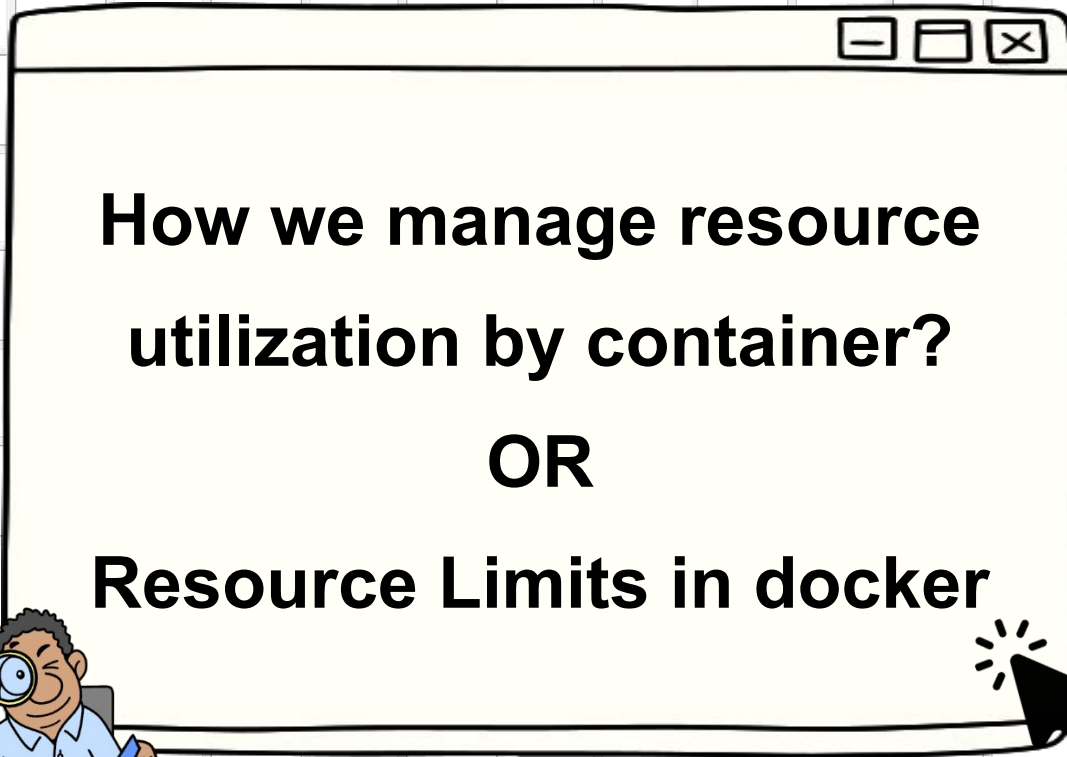


Use .dockerignore

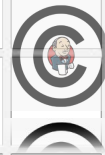
Excludes unnecessary files from being copied into the image.



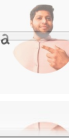
Karan Gupta



Achieving Container Efficiency



Karan Gupta



Performance Management



Enhances performance through effective resource allocation.

Cost Management



Reduces costs by optimizing resource consumption.

Resource Prevention



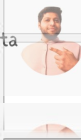
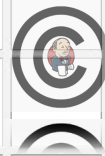
Stops containers from monopolizing host resources.



System Stability

Maintains stability in multi-container setups.





Memory Limits:

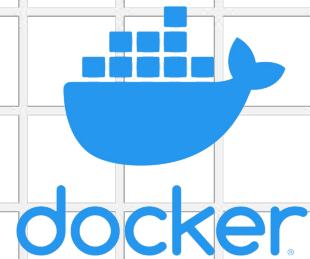
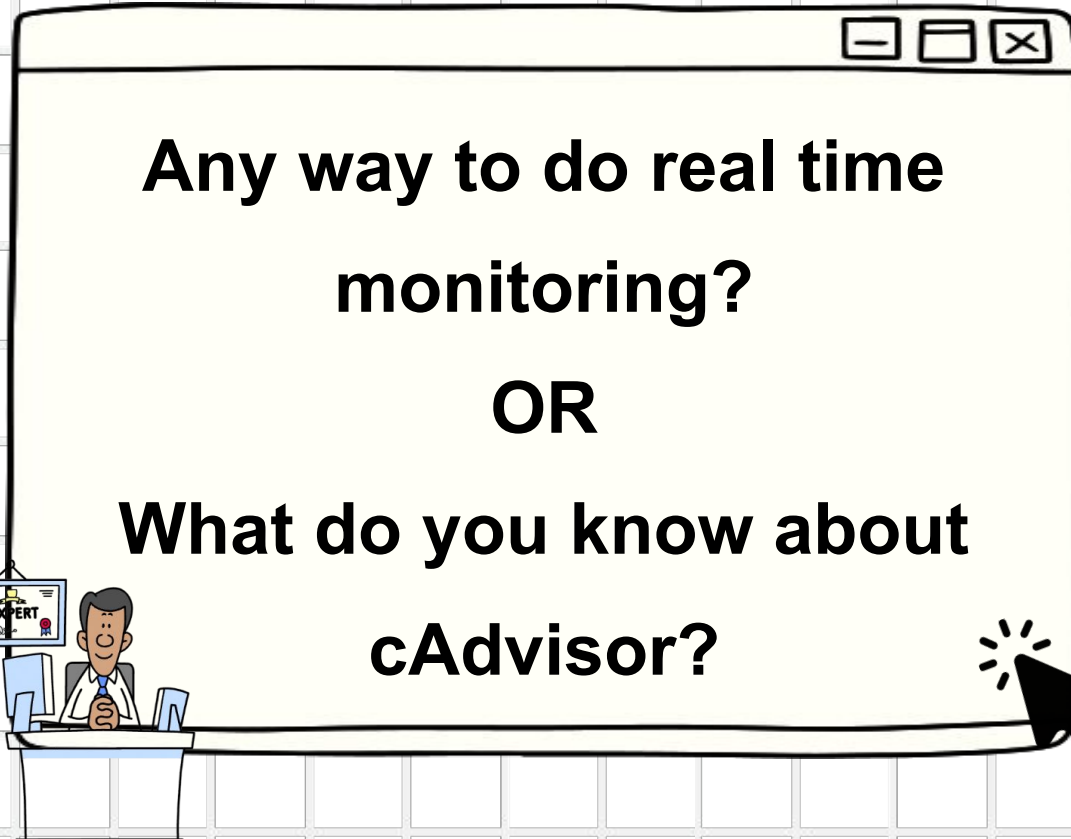
Type	Description	Docker Option
Soft Limit	Preferred max usage; can exceed if available	<code>--memory-reservation</code>
Hard Limit	Absolute max; container is killed if exceeded	<code>--memory, -m</code>

CPU Limits:

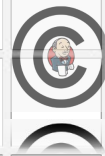
Type	Description	Docker Option
Soft Limit	Relative priority in shared CPU environments	<code>--cpu-shares</code>
Hard Limit	Absolute limit (e.g., 1.5 cores max)	<code>--cpus</code>



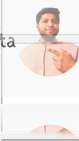
Karan Gupta



cAdvisor



Karan Gupta



- **cAdvisor (Container Advisor)** is an open-source tool by Google.
- Provides **real-time monitoring and performance analysis** of containers.

cAdvisor Features



Resource Usage

Tracks CPU, memory, disk, and network usage.

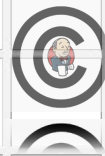
Automatically captures metrics at the container level.

Container Metrics

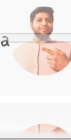


Web UI

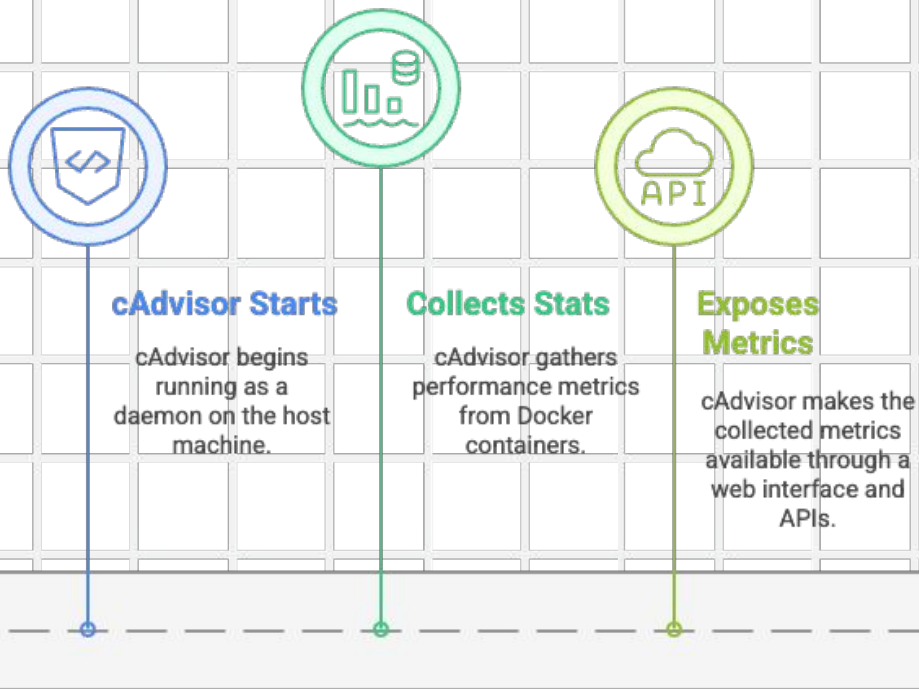
Offers a web user interface for live monitoring.



Karan Gupta

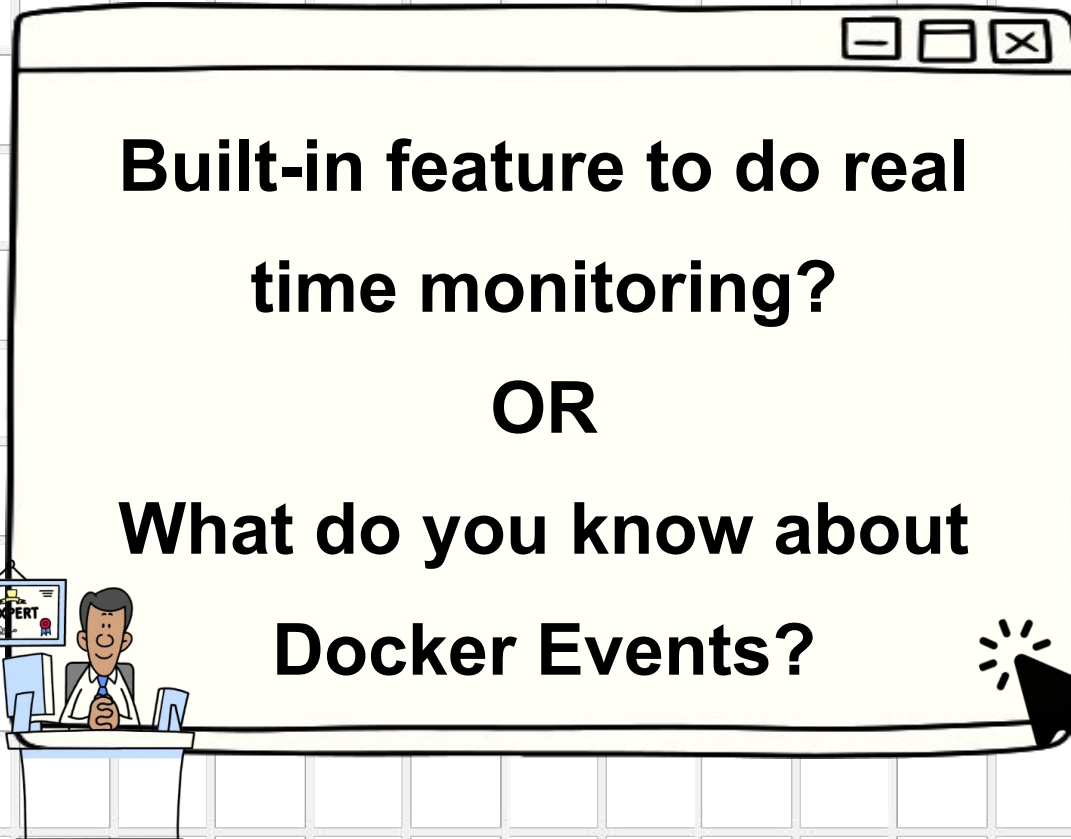


cAdvisor Operational Sequence





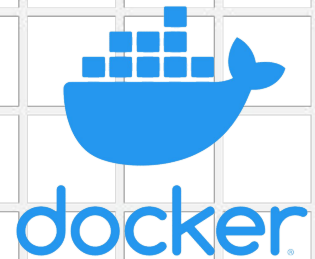
Karan Gupta



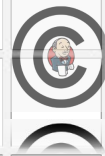
**Built-in feature to do real
time monitoring?**

OR

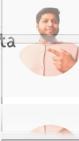
**What do you know about
Docker Events?**



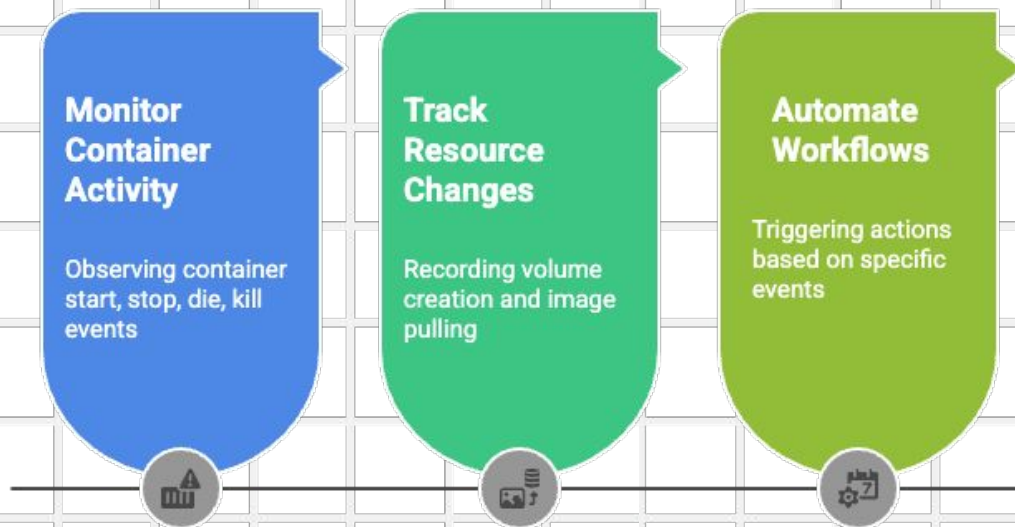
Docker Events

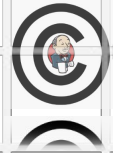


Karan Gupta



- Docker events are **real-time notifications** from the Docker daemon.
- They report **lifecycle activities** of Docker objects such as containers, networks, volumes, and images.

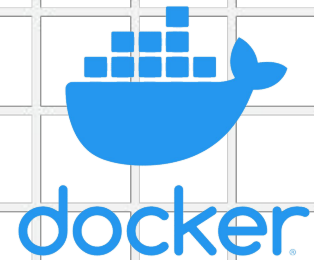




Karan Gupta



Docker Restart Policy

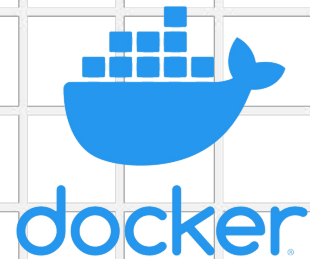




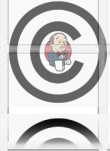
Karan Gupta



**Your container stops
immediately after running.
What steps do you take to
debug the issue?**



Debugging Docker Containers



Karan Gupta



Temporarily delay the container to aid debugging.



Determine if the container exited successfully.



Explore the container manually using ``bash`` or ``sh``.



Ensure the container has a long-running process.



Examine container details for configuration problems.



Review container logs for errors or issues.

1

2

3

4

5

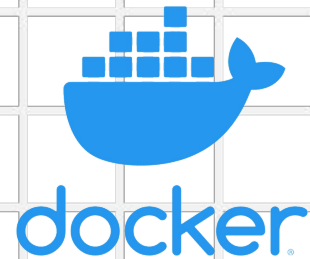
6

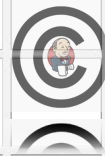


Karan Gupta

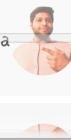


**How do you clean up
unused containers, images,
and volumes in a production
environment?**





Karan Gupta



Docker Resource Cleanup Process

Remove Stopped Containers



Eliminates containers that are no longer running

Remove Unused Volumes

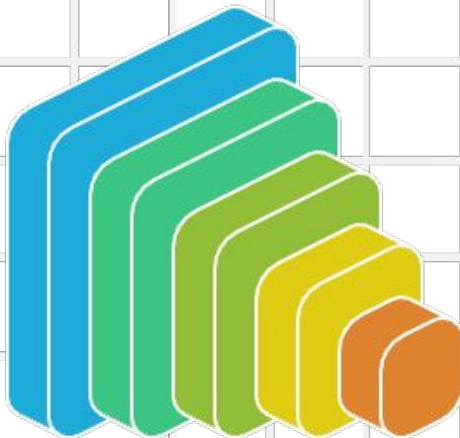


Cleans up volumes not attached to containers

All-in-One Cleanup



Executes a comprehensive cleanup of all resources



Remove Unused Images

Deletes images not in use by any container

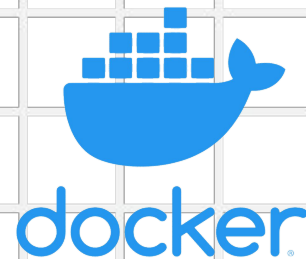
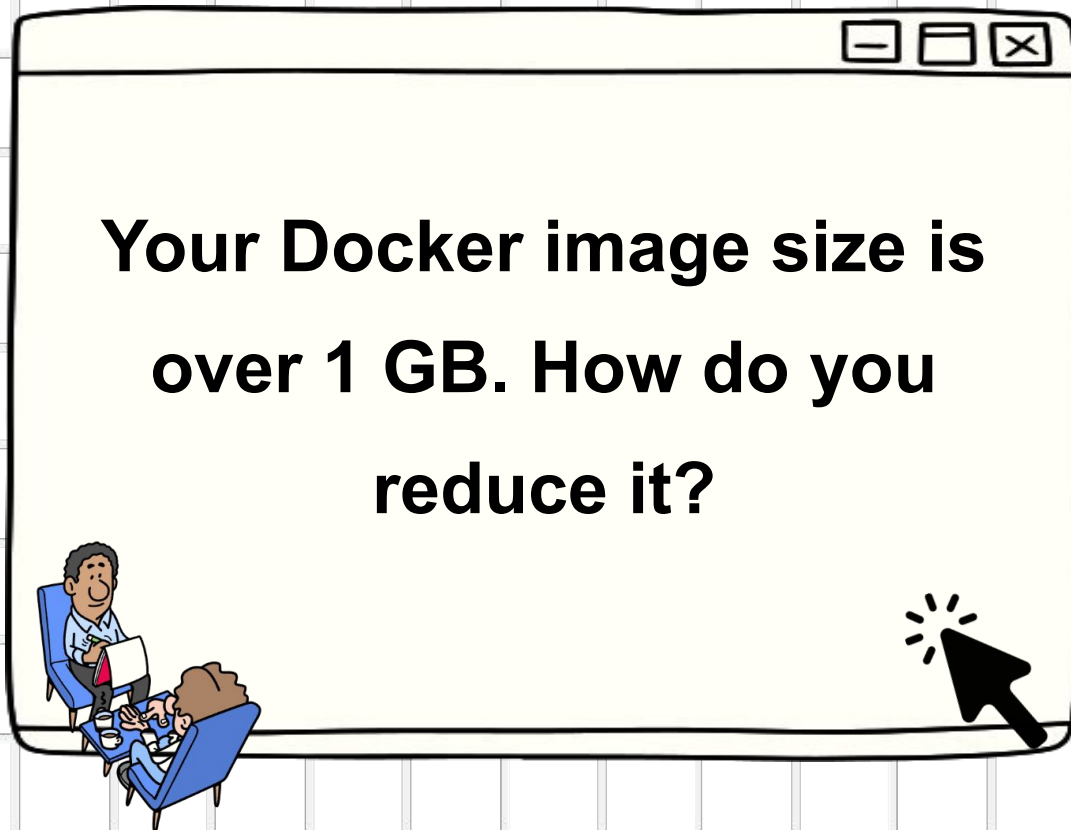


Remove Unused Networks

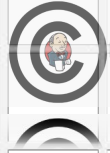
Removes networks not connected to containers



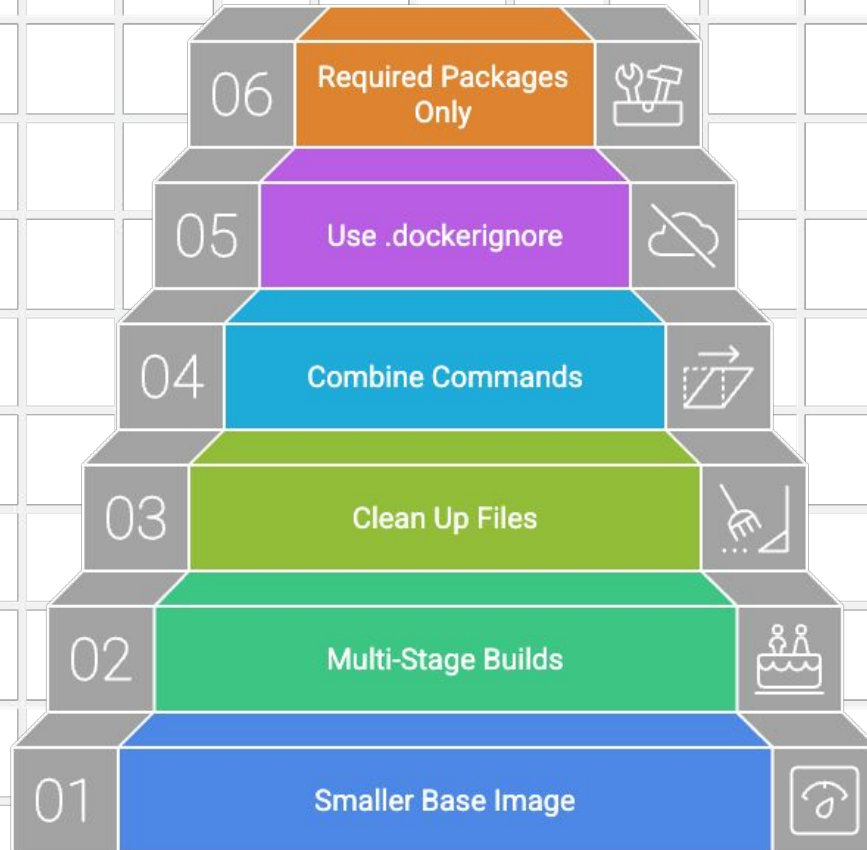
Karan Gupta

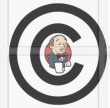


Steps to Reduce Docker Image Size



Karan Gupta

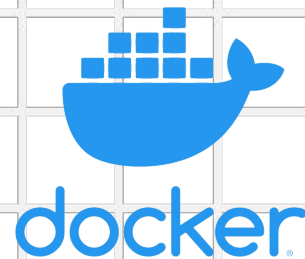




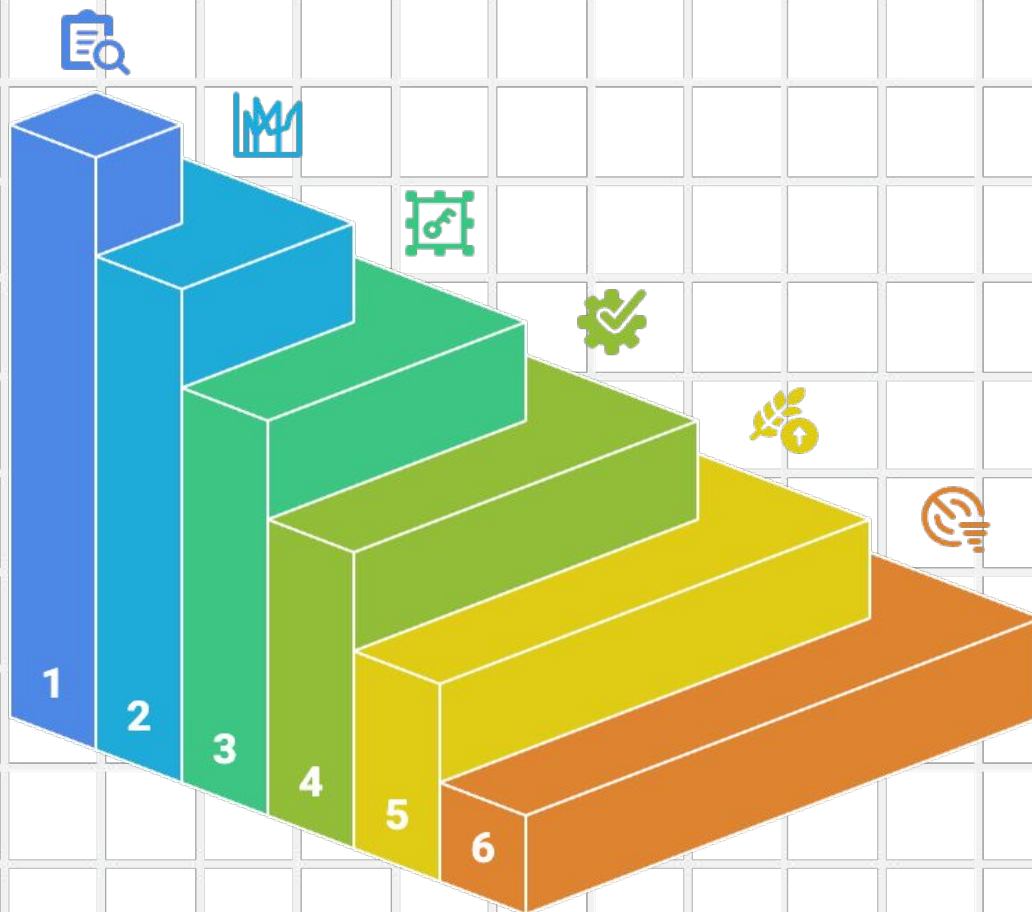
Karan Gupta



**A container uses too much
memory and gets killed. How
will you fix this?**



Steps to Resolve Container Memory Issues



Confirm Memory Issue

Verify if the container is being killed due to memory issues by checking logs and exit reasons.

Analyze Memory Usage

Use ``docker stats`` to monitor memory usage in real-time and identify patterns.

Set Memory Limits

Limit the container's memory usage to prevent host overload.

Optimize Application

Identify and fix memory leaks or high memory-consuming operations.

Use Smaller Images

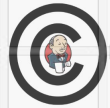
Reduce memory footprint by using lightweight base images.

Monitor with Tools

Track container memory usage over time using external tools like cAdvisor.



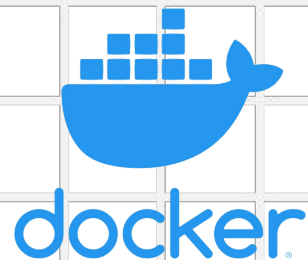
Karan Gupta



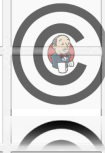
Karan Gupta



**How do you scan Docker
images for vulnerabilities?**



Container Image Scanning Tools



Karan Gupta

Docker Scout

Integrated into Docker Desktop for quick image analysis.



Snyk CLI

Command-line tool for testing container images for vulnerabilities.



GitLab Scanning

Automated container scanning supported through GitLab CI/CD pipelines.



Trivy

Supports scanning of file systems, Git repositories, and containers.



Docker Hub Scans

Enable automated vulnerability scanning for repositories on Docker Hub.



CI/CD Integration

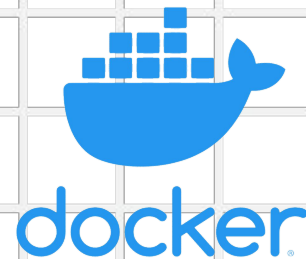
Embed security scans into CI/CD pipelines using tools like Trivy.



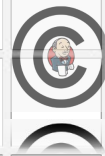
Karan Gupta



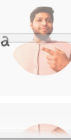
**How do you check how
many times a container was
restarted and why?**



Docker Container Troubleshooting Sequence



Karan Gupta



Monitor Docker Events

Observe real-time events for insights into container behavior



View Container Logs

Examine logs for error messages or unusual activity



Check Exit Status and OOM Kill Flag

Determine if the container exited due to memory issues



Check Restart Count

Use ``docker inspect`` to view restart history

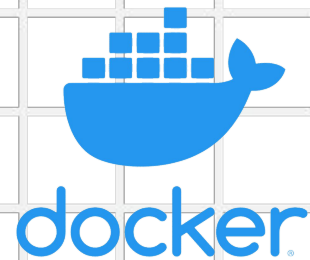




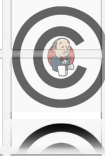
Karan Gupta



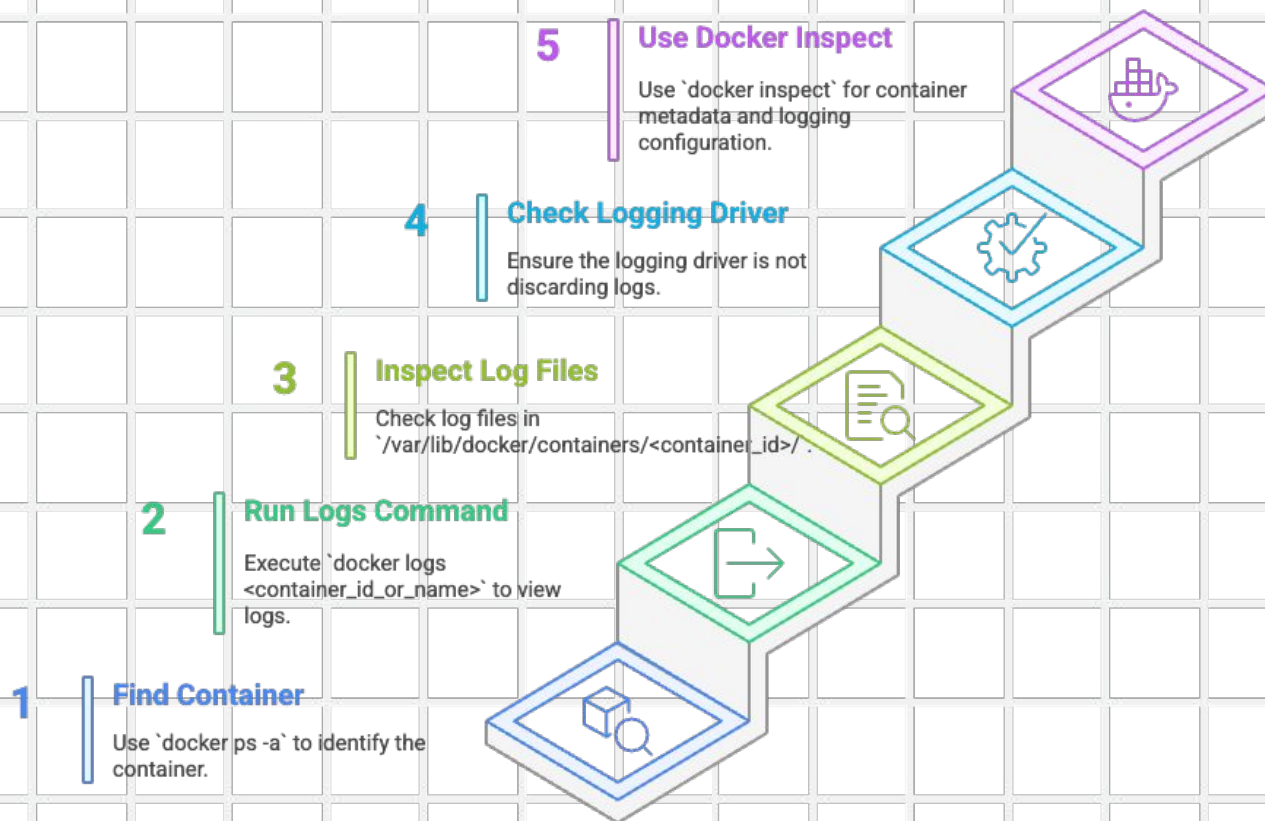
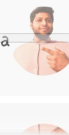
**How do you inspect container
logs when it has already
stopped?**



Inspecting Docker Container Logs

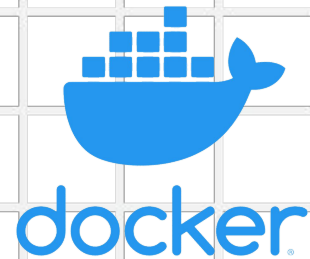
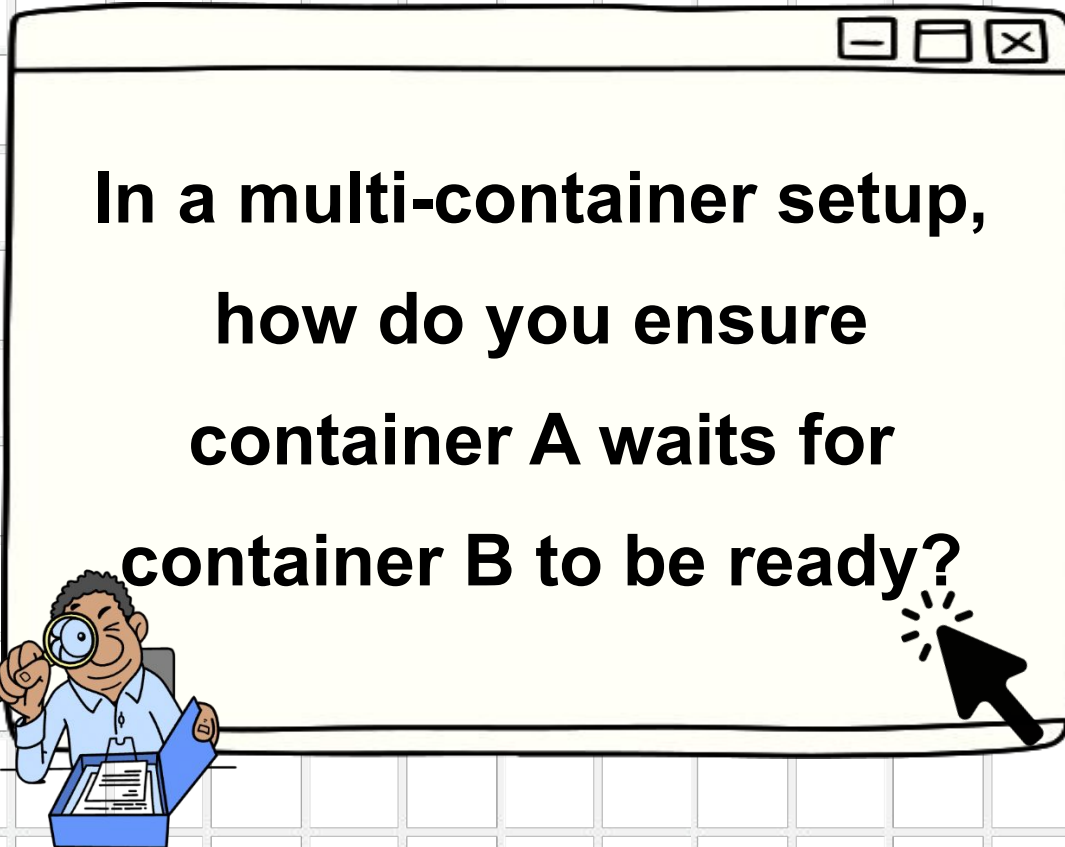


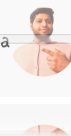
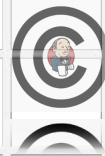
Karan Gupta





Karan Gupta





How to ensure container dependencies are met?



Use ``depends_on``

Ensures startup order but not readiness.



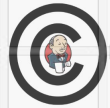
Implement Wait Script

Checks readiness before proceeding.



Use Health Checks

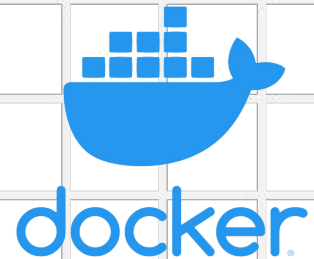
Monitors container health and readiness.

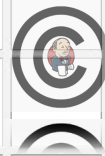


Karan Gupta

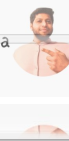


**How do you track which
containers are consuming
the most resources in
real-time?**

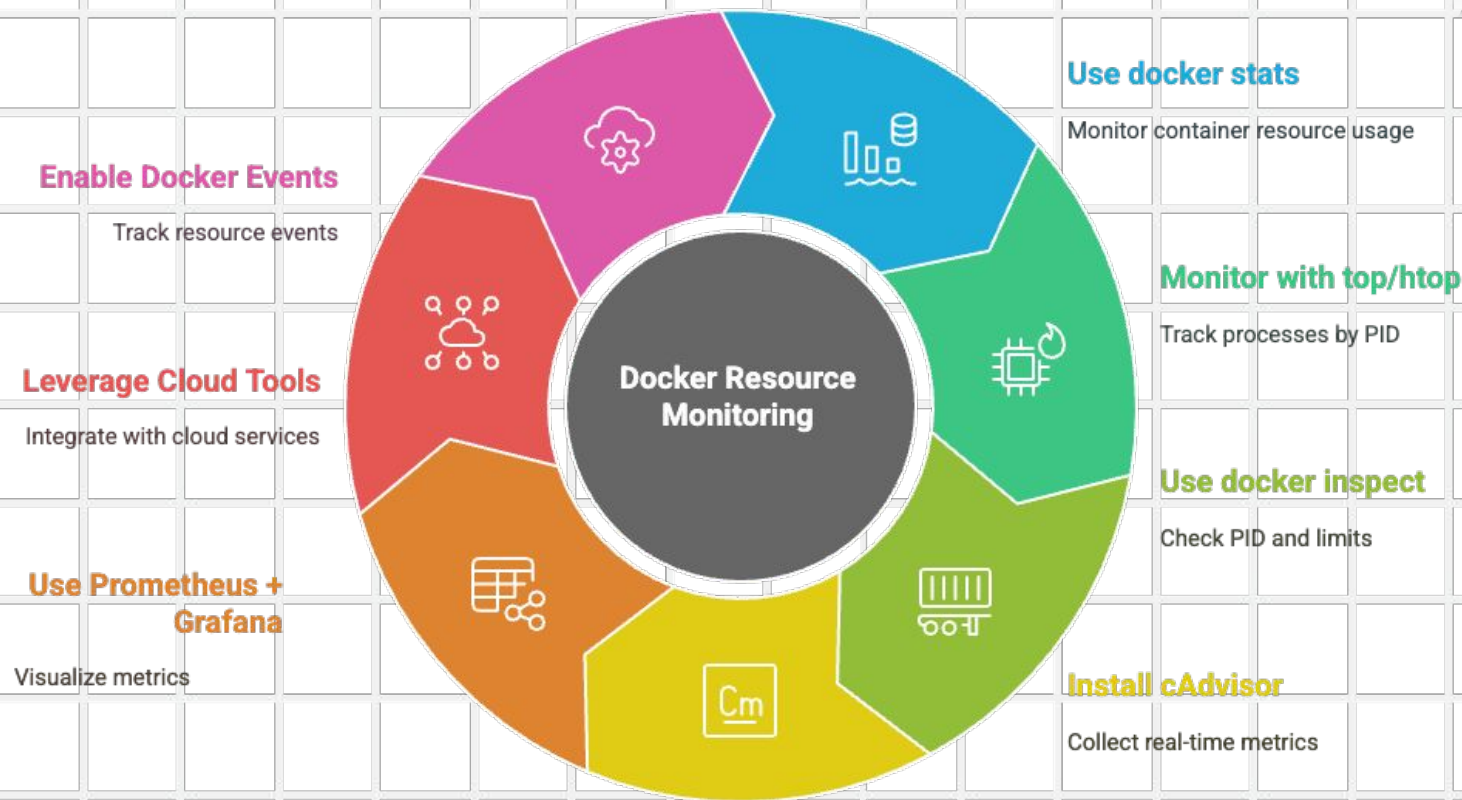


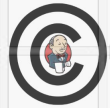


Karan Gupta

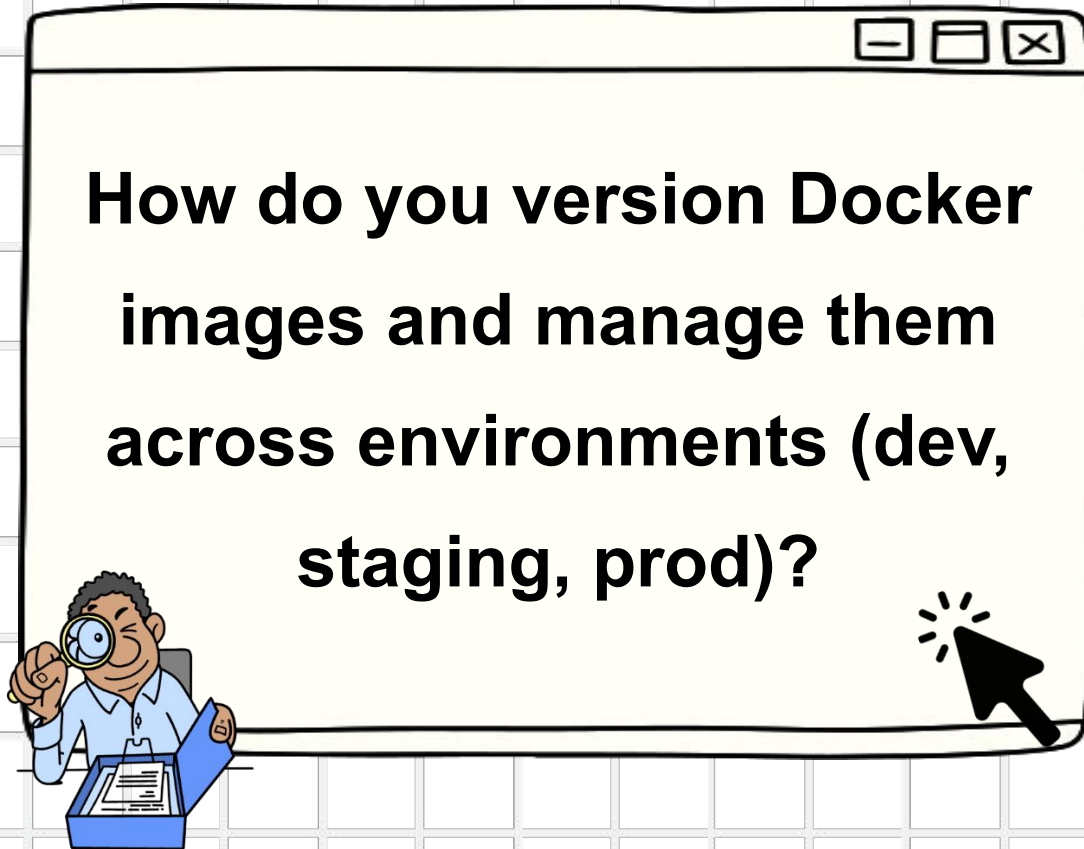


Docker Resource Monitoring Cycle

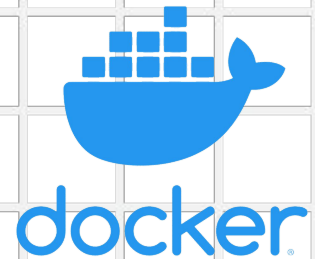


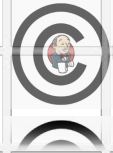


Karan Gupta



**How do you version Docker
images and manage them
across environments (dev,
staging, prod)?**





Karan Gupta



Effective Docker Image Management

Semantic Versioning

Using structured versioning for clarity and consistency

Environment Tagging

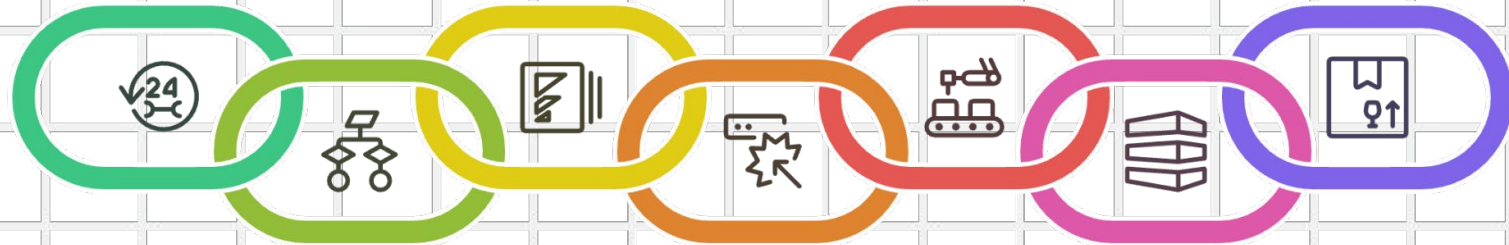
Differentiating images by environment for deployment

CI/CD Automation

Automating tagging through CI/CD pipelines

Immutable Promotion

Promoting images with immutable tags for stability



Git Integration

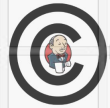
Linking images to specific Git commits for traceability

Latest Tag Usage

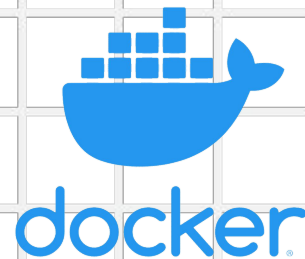
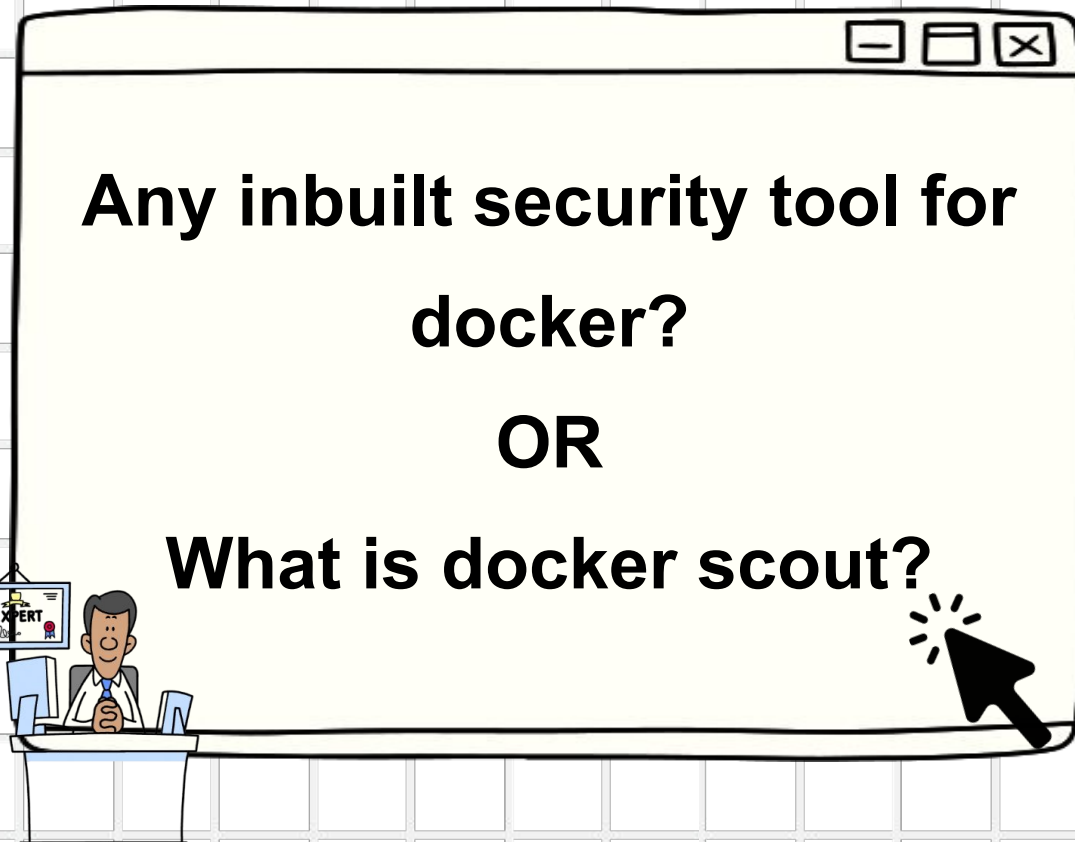
Maintaining a 'latest' tag for development purposes

Central Registry

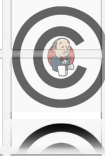
Storing images in a central registry for accessibility



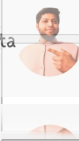
Karan Gupta



Docker Scout

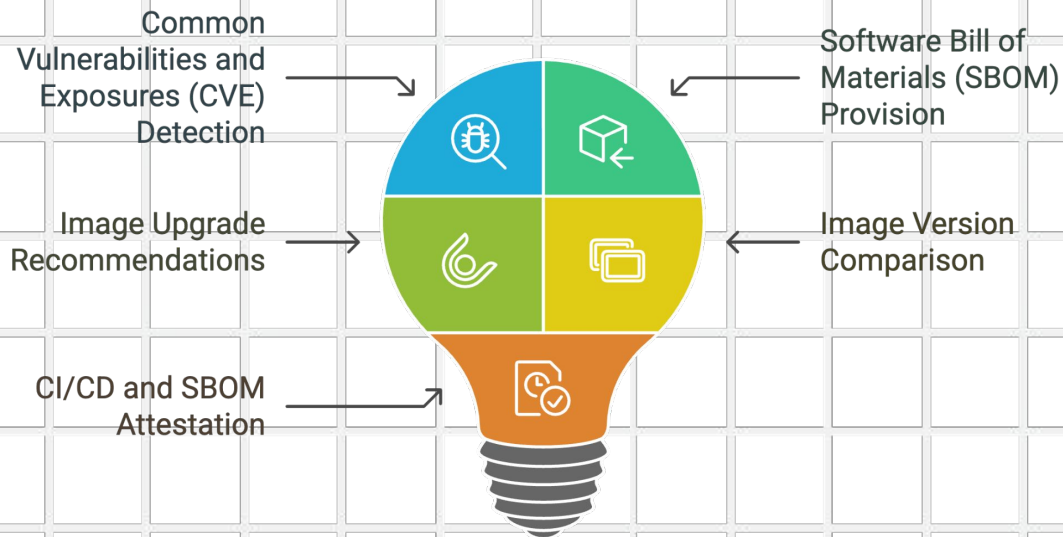


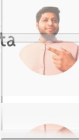
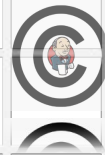
Karan Gupta



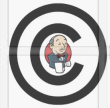
- A security tool from Docker to scan container images for vulnerabilities and analyze image content.

Docker Scout Features Overview





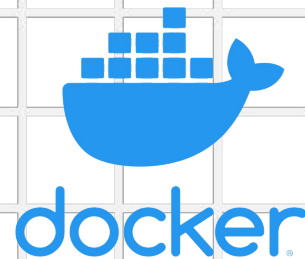
Command	Purpose
<code>docker scout cves <image></code>	Scan image for vulnerabilities
<code>docker scout sbom <image></code>	Generate Software Bill of Materials
<code>docker scout compare <img1> <img2></code>	Compare two images (layers, packages, CVEs)
<code>docker scout recommendations <image></code>	Suggest image upgrades and fixes
<code>docker scout attestation generate</code>	Generate metadata for secure supply chain
<code>docker scout attestation push</code>	Push scan results & attestations to registry



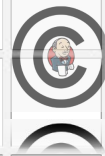
Karan Gupta



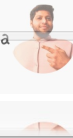
**Have you ever used Trivy for
scanning purpose?**



Trivy



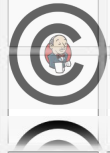
Karan Gupta



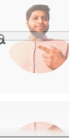
Trivy (by Aqua Security) is a **simple and comprehensive vulnerability scanner** for:

- Docker images
- File systems
- Git repositories
- Kubernetes resources
- SBOMs (Software Bill of Materials)

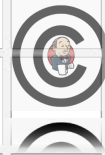
Command	Purpose
<code>trivy image nginx:latest</code>	Scan Docker image for CVEs
<code>trivy image --severity HIGH,CRITICAL myapp:latest</code>	Focus on critical issues only



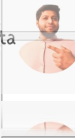
Karan Gupta



Security scanning features



Karan Gupta



OS package vulnerabilities

Scans for operating system package vulnerabilities.

Scans for language-specific dependencies such as npm, pip, and Maven.

Language-specific dependencies



Misconfigurations detection

Detects misconfigurations in Dockerfiles, Kubernetes YAMLs, and Terraform.

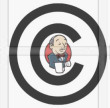
Supports Software Bill of Materials generation.

SBOM generation



CI/CD ready

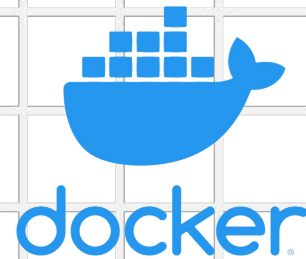
CLI-friendly and ready for Continuous Integration and Continuous Delivery pipelines.



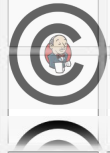
Karan Gupta



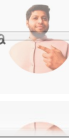
**What do you understand by
logging driver in docker?**



Logging Driver



Karan Gupta



- ❑ Mechanism to control how container logs are captured and where they're sent.
- ❑ By default, Docker uses the `json-file` driver.

- **Why Important?**

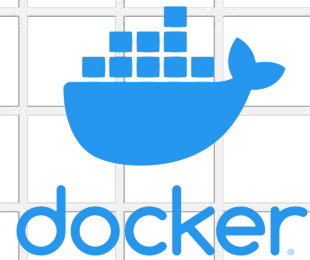
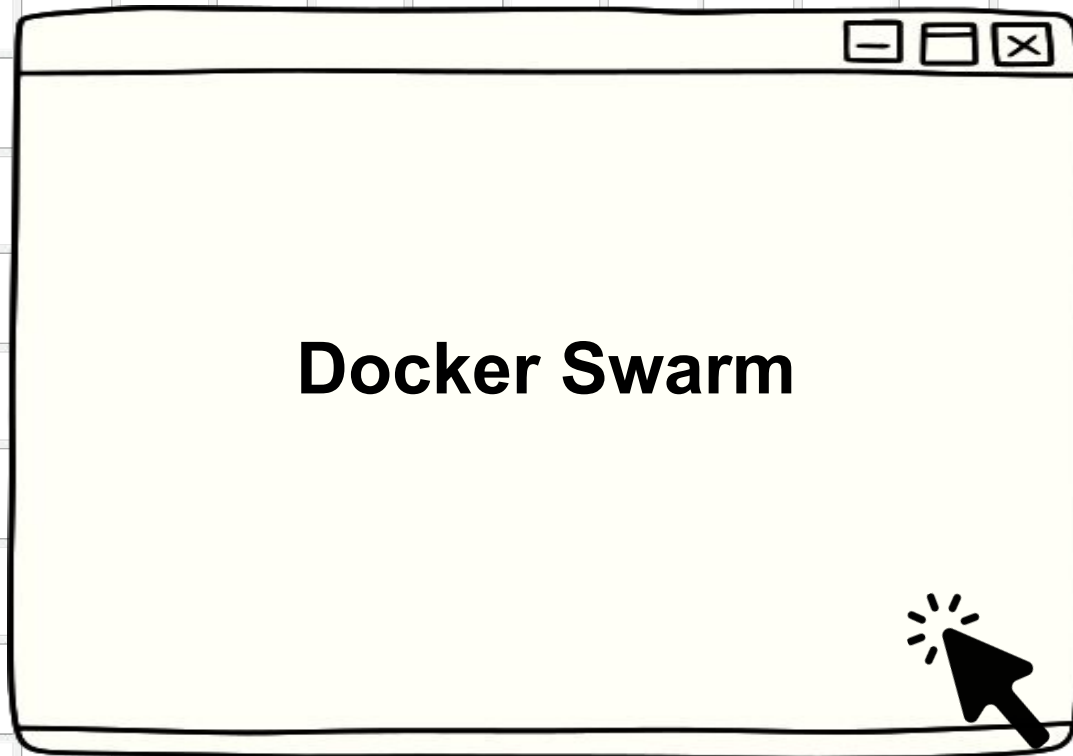
- a. Supports centralized log management.
- b. Integrates with external systems for monitoring and auditing.

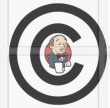
- **Common Use Cases:**

- a. Debugging containers
- b. Shipping logs to ELK, Splunk, or CloudWatch
- c. Real-time log analysis



Karan Gupta

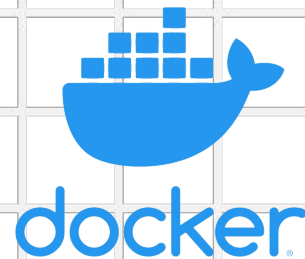


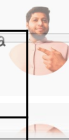


Karan Gupta

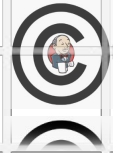


**Any information on other types
of Network?**





Network	Overlay	Macvlan	IPvlan
Purpose	Multi-host communication (Swarm/K8s)	Assign real MAC addresses to containers	Lightweight L2 networking with IP separation
Host Requirement	Requires multiple Docker hosts (or Swarm mode)	Works only on same host	Same host, shares parent interface
Communication Scope	Across multiple hosts	Container ↔ external LAN	Container ↔ external LAN
MAC Address Handling	Virtual MACs handled via tunneling	Each container gets a unique MAC address	All containers share the parent interface's MAC address
IP Address Handling	Virtual IPs via internal DNS	IP assigned from physical network	IP assigned from physical network
Use Case Examples	Swarm/Kubernetes services	Legacy apps, network appliances needing real IPs	High-performance containers with strict network policy

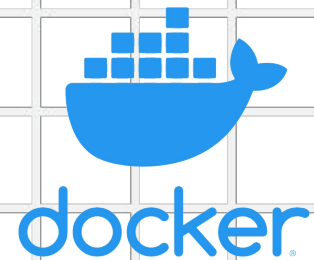


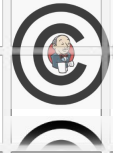
Karan Gupta



Docker Installation

AWS

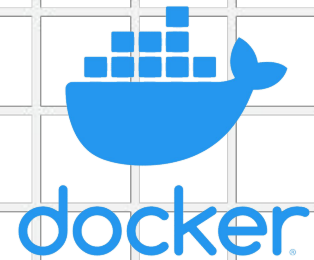


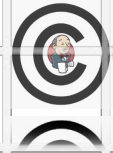


Karan Gupta

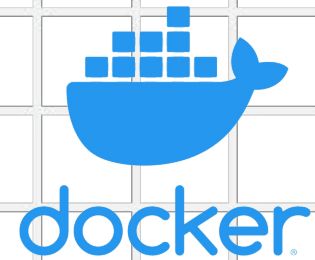
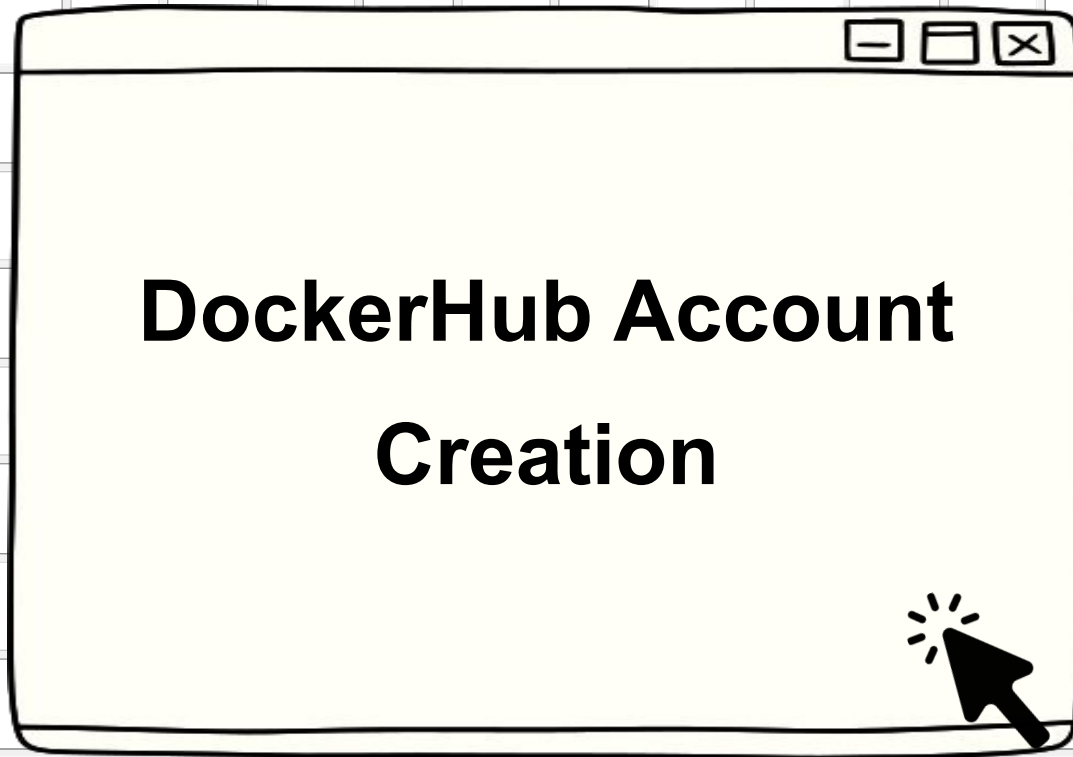


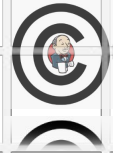
Docker Container Hello-World



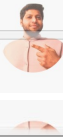


Karan Gupta

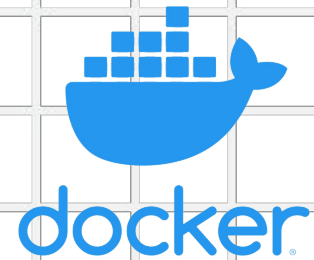


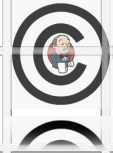


Karan Gupta

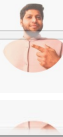


Docker Image Commands - I

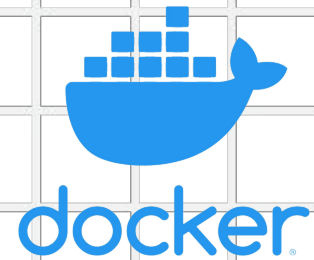


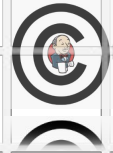


Karan Gupta

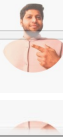


Docker Container Commands - II

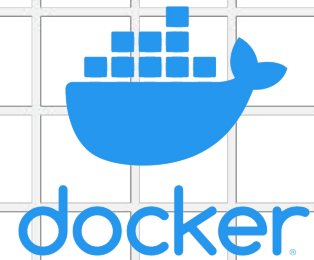


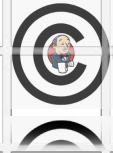


Karan Gupta



Docker Container Commands - III

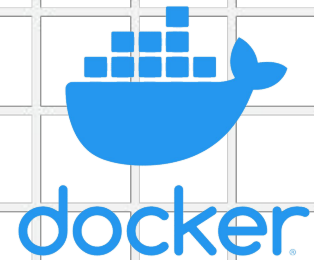


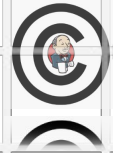


Karan Gupta



Docker Container Commands - IV

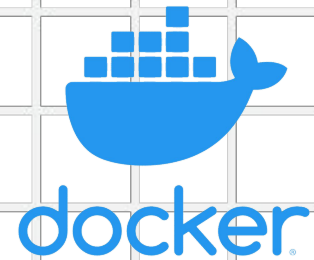


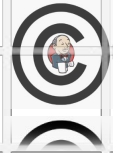


Karan Gupta



Docker Container Commands - V

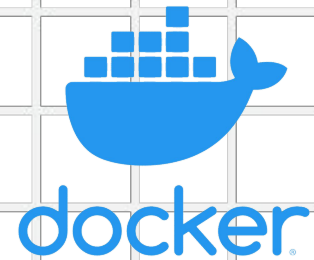


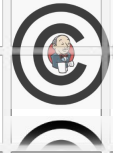


Karan Gupta

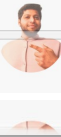


Getting inside a container

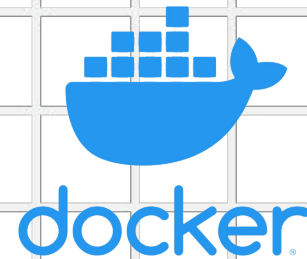


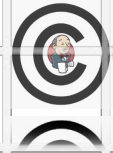


Karan Gupta

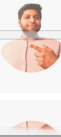


**Is it possible to run a
container with a cmd?**

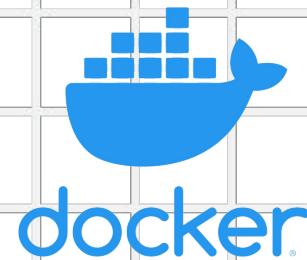


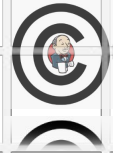


Karan Gupta

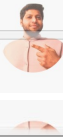


**How to bring container
in foreground?**

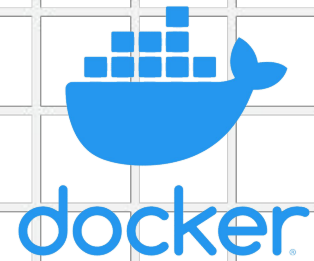


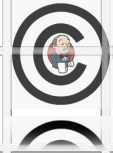


Karan Gupta

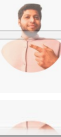


Prune Command

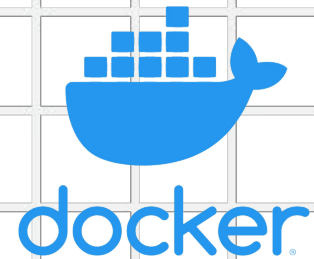


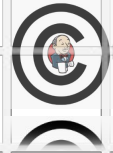


Karan Gupta



Create vs Run vs Start vs Restart

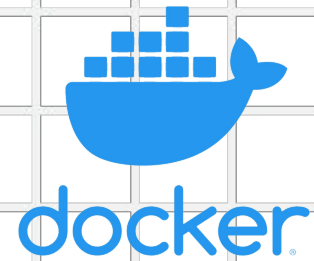


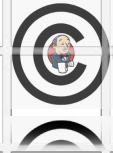


Karan Gupta



Export & Import

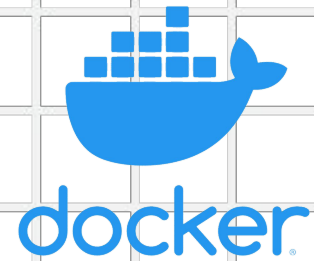


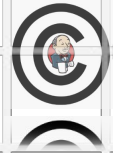


Karan Gupta

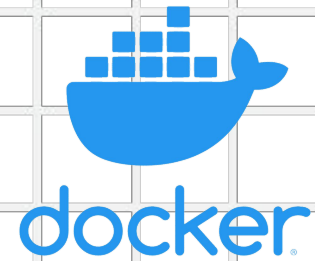
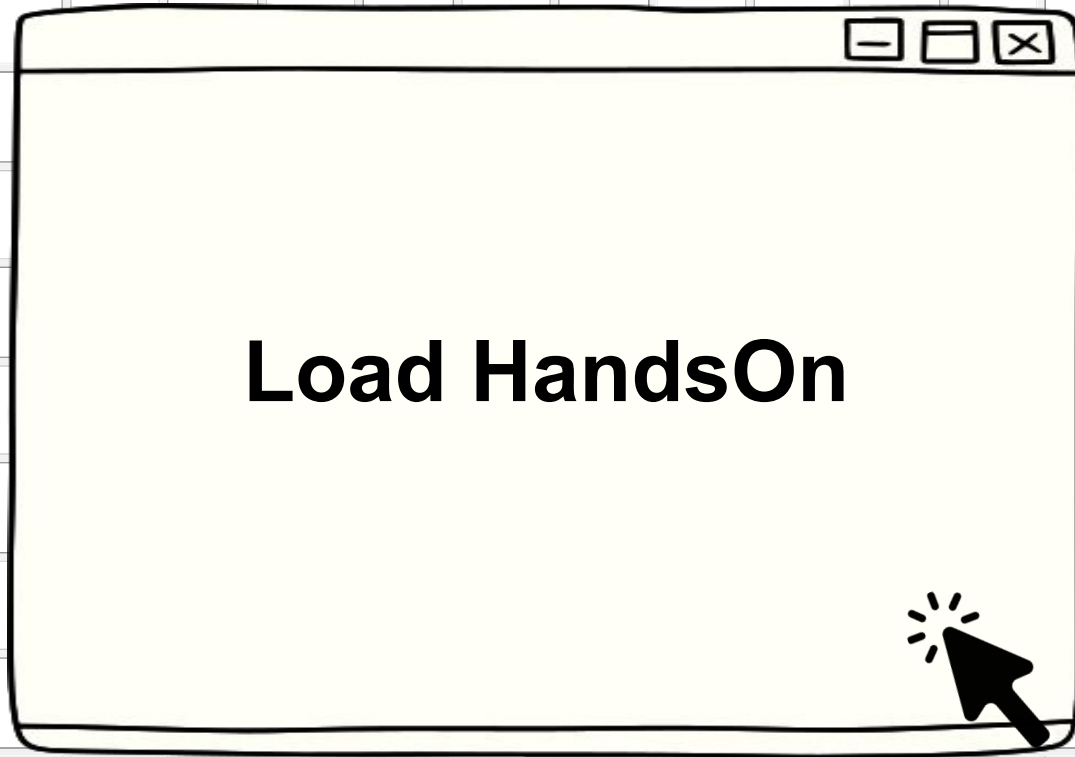
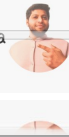


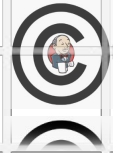
Save & Load



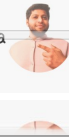


Karan Gupta

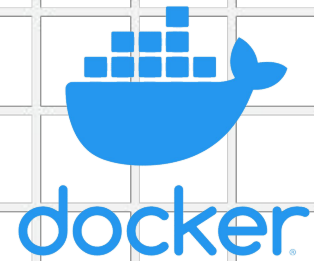


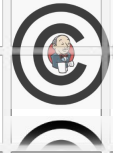


Karan Gupta



Copy & Diff

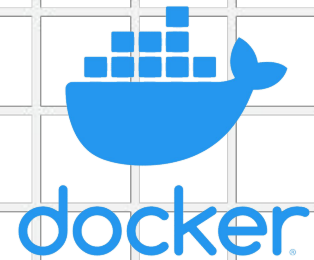


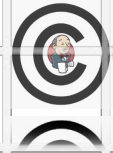


Karan Gupta



Statistics of Containers

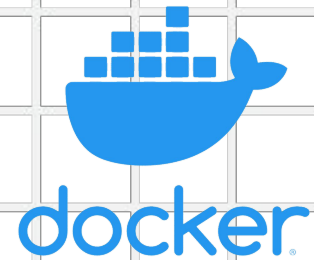


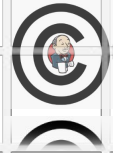


Karan Gupta



**How to access
applications?**

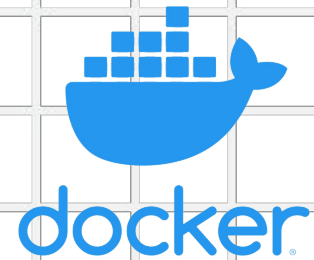


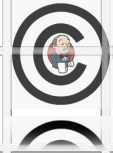


Karan Gupta



**How to access
applications on browser?**

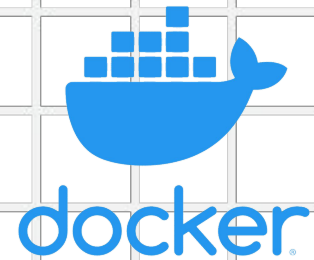


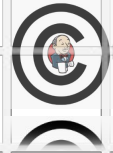


Karan Gupta



Docker Login

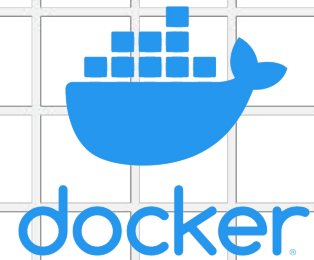


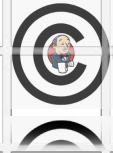


Karan Gupta

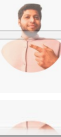


How to run Docker Container in Background?

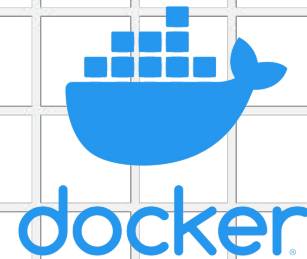


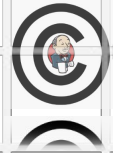


Karan Gupta



**Can I remove the image of
a running container?**

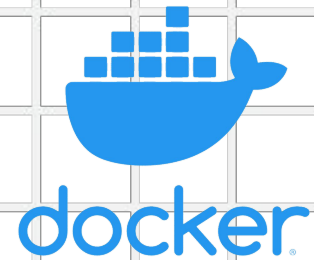


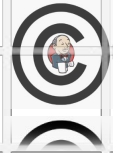


Karan Gupta

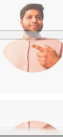


Killing a container
Exited (137)

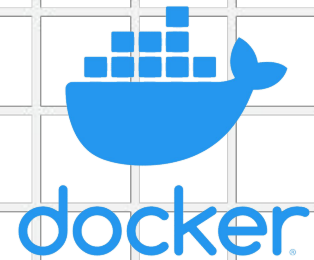


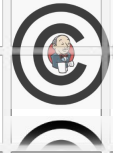


Karan Gupta



Docker Image Commands - II

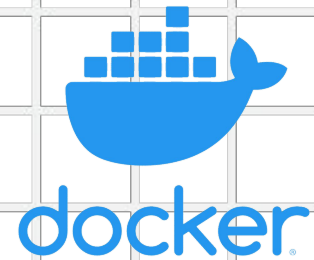


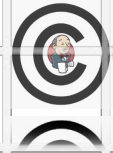


Karan Gupta



Docker Image Commands - III

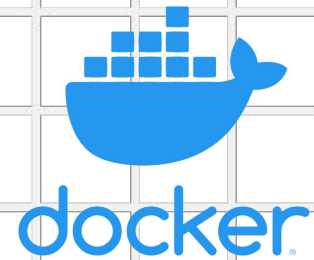


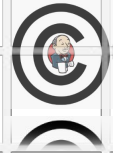


Karan Gupta



Logging Driver AWS Cloudwatch

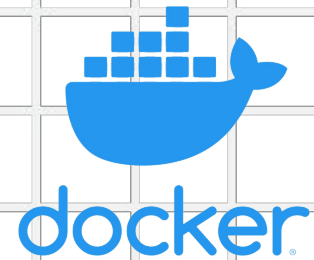


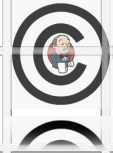


Karan Gupta



Basic Dockerfile - II

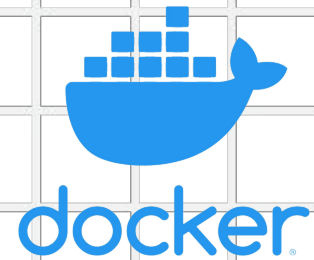


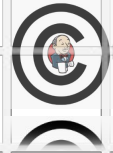


Karan Gupta



Basic Dockerfile - III

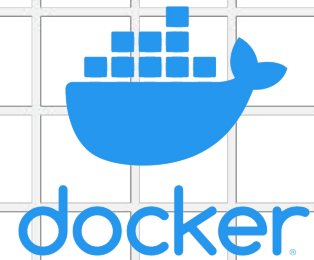


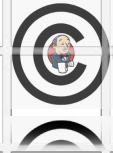


Karan Gupta



Basic Dockerfile - IV

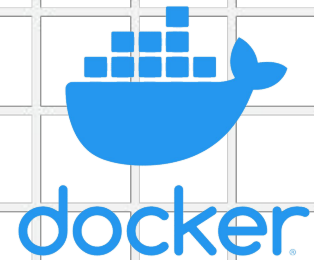


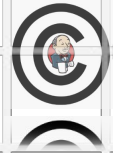


Karan Gupta

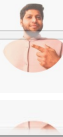


Basic Dockerfile - V

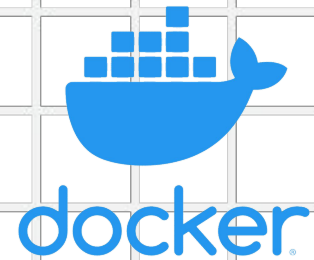


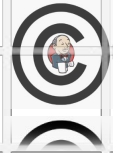


Karan Gupta



Basic Dockerfile - VI

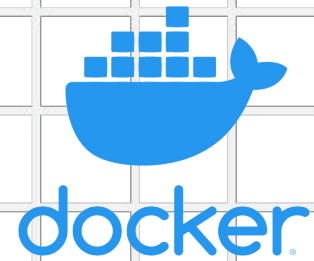


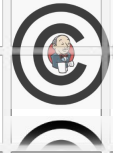


Karan Gupta

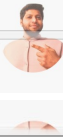


COPY vs ADD

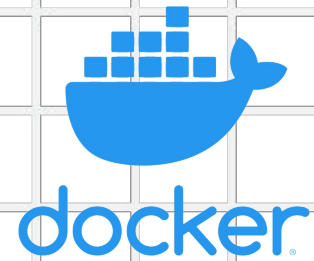


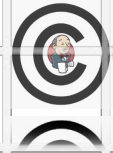


Karan Gupta

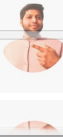


Dockerfile Path

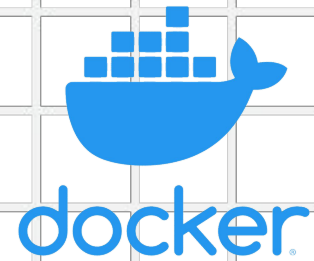


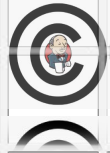


Karan Gupta

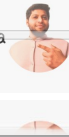


Setting Context

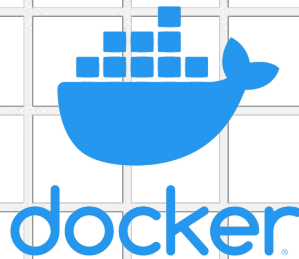


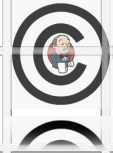


Karan Gupta

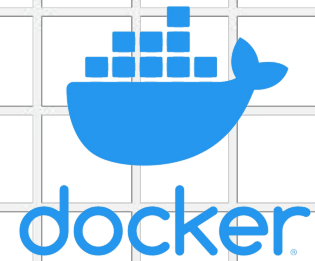
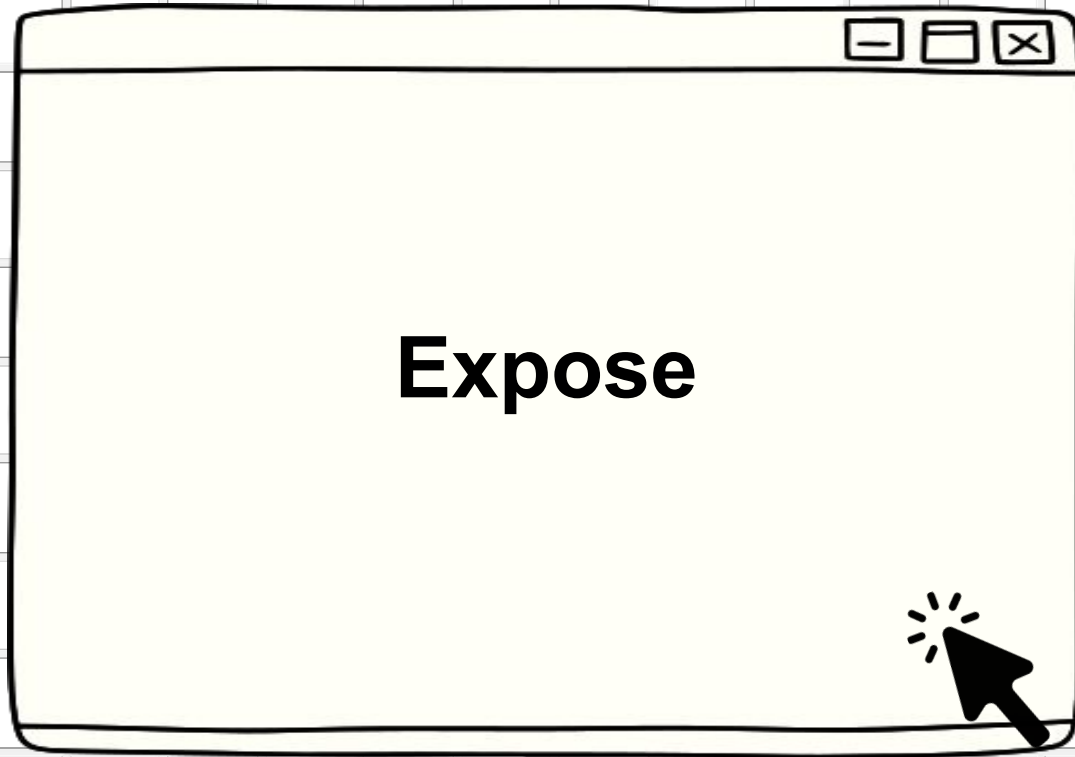
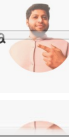


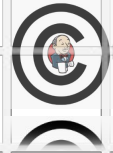
Entrypoint





Karan Gupta

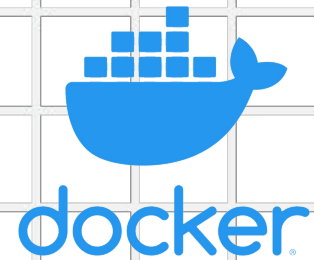


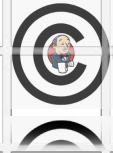


Karan Gupta



Removing Container Automatically

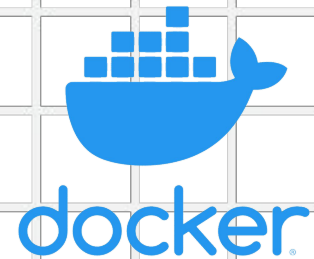


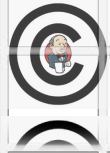


Karan Gupta



Running External Script in Dockerfile

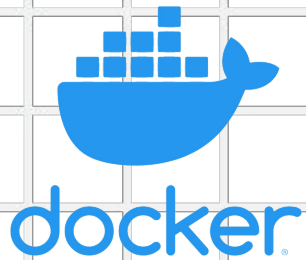


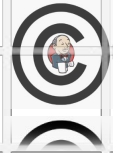


Karan Gupta

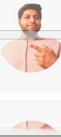


Default Storage

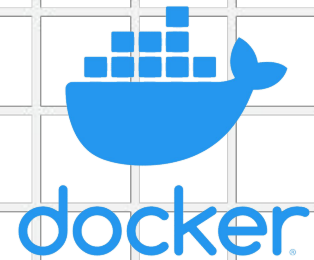


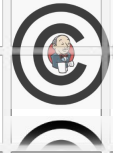


Karan Gupta

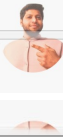


Docker Volume (Anonymous)

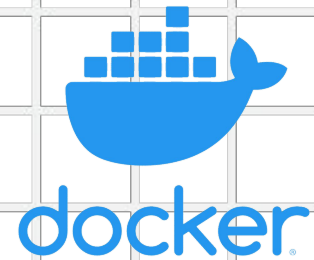


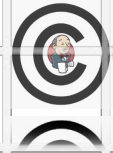


Karan Gupta



Docker Volume (Read-Only)

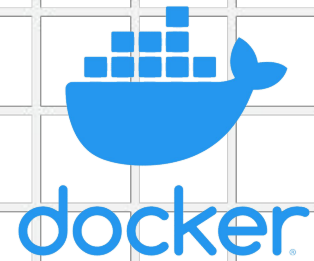


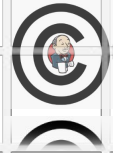


Karan Gupta

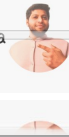


Deleting Volume

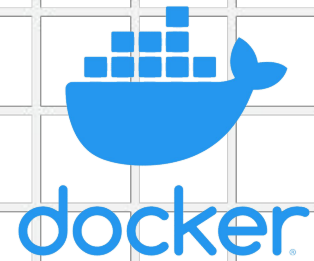


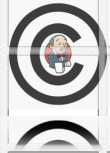


Karan Gupta

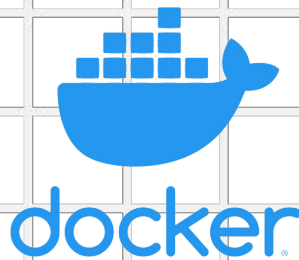
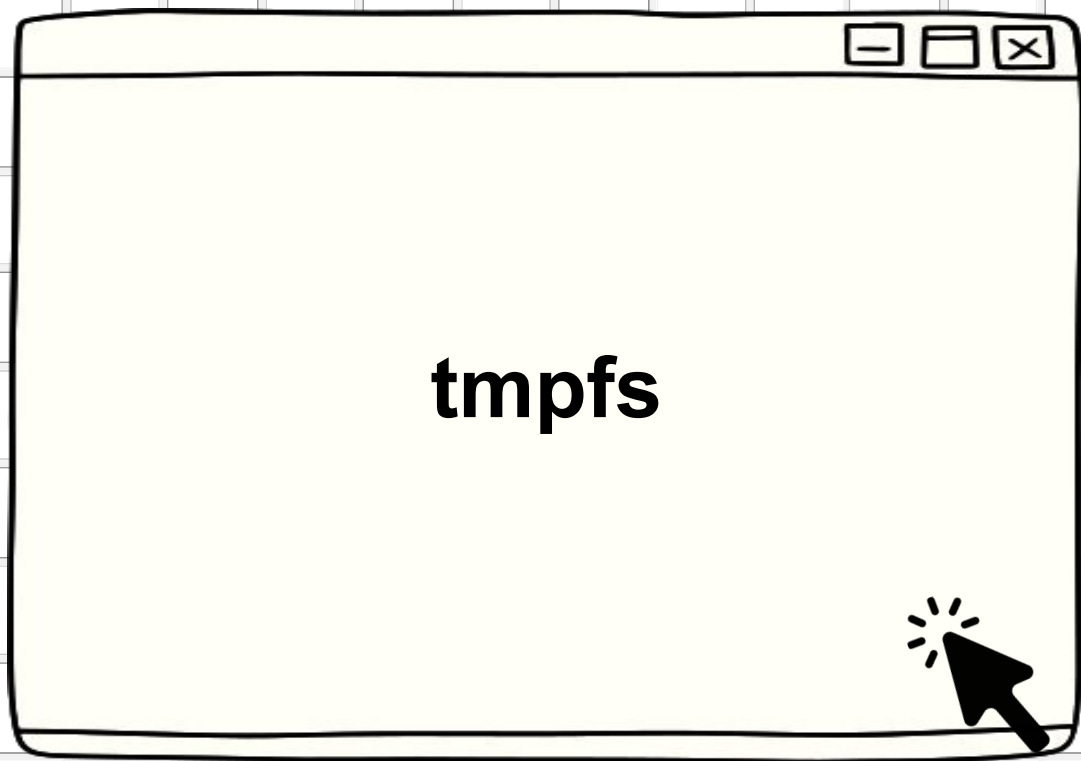
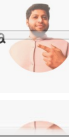


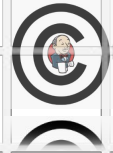
Bind Mount



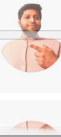


Karan Gupta

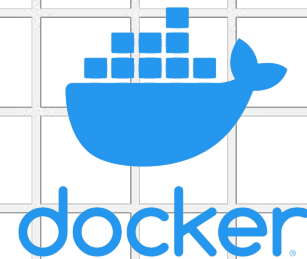


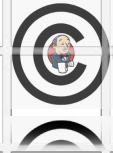


Karan Gupta



**Testing site without
rebuilding image?**

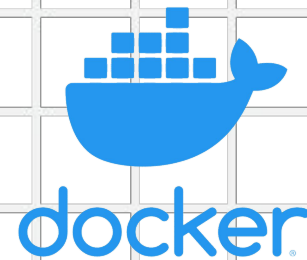


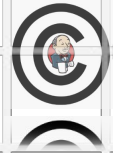


Karan Gupta



**Separate Folder for
each container?**

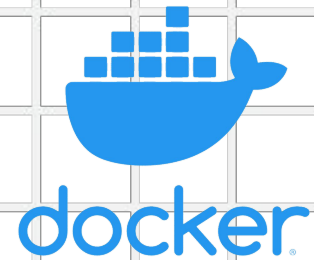


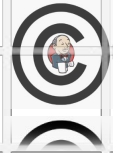


Karan Gupta



Using EFS as Storage





Karan Gupta

